

Fully Adaptive Schnorr Threshold Signatures

Elizabeth Crites¹, Chelsea Komlo², and Mary Maller³

¹ Web3 Foundation

² University of Waterloo & Zcash Foundation

³ Ethereum Foundation & PQShield, UK

elizabeth@web3.foundation, ckomlo@uwaterloo.ca, mary.maller@ethereum.org

Abstract. We prove adaptive security of a simple three-round threshold Schnorr signature scheme, which we call **Sparkle+**. The standard notion of security for threshold signatures considers a *static* adversary – one who must declare which parties are corrupt at the beginning of the protocol. The stronger *adaptive* adversary can at any time corrupt parties and learn their state. This notion is natural and practical, yet not proven to be met by most schemes in the literature.

In this paper, we demonstrate that **Sparkle+** achieves several levels of security based on different corruption models and assumptions. To begin with, **Sparkle+** is statically secure under minimal assumptions: the discrete logarithm assumption (DL) and the random oracle model (ROM). If an adaptive adversary corrupts fewer than $t/2$ out of a threshold of $t + 1$ signers, then **Sparkle+** is adaptively secure under a weaker variant of the one-more discrete logarithm assumption (AOMDL) in the ROM. We then prove that **Sparkle+** achieves *full* adaptive security, with a corruption threshold of t , under AOMDL in the algebraic group model (AGM) with random oracles. Ours is the *first* proof for a threshold Schnorr signature scheme achieving the following desiderata: (i) adaptive security for a full t out of $t + 1$ signers, for any $t < n$, (ii) no requirement of secure erasure of secret state, and (iii) less than $\binom{n}{t}$ security loss. More than that, our reduction for full adaptive security is tight. We achieve this using minimal network assumptions and only two online signing rounds with pre-processing.

Update: Recent work [33] shows that the adaptive security of **Sparkle+**, among other threshold Schnorr schemes, cannot hold for t corruptions without assuming the hardness of a search problem P defined in [33]. (The $t/2$ adaptive security of **Sparkle+** is unaffected.) Later work [31] shows the same for a closely related problem, LDVR, defined in [31]. The FROST threshold Schnorr signature scheme is proven adaptively secure for t corruptions under the LDVR assumption [31]. We leave it to future work to explore whether **Sparkle+** can be proven adaptively secure for t corruptions using similar techniques.

Table of Contents

1	Introduction	3
2	Related Work	6
3	Preliminaries	10
3.1	General Notation and Definitions	10
3.2	Digital Signatures	12
3.3	Assumptions	13
4	Threshold Signature Schemes	15
4.1	Definition of Static Security	18
4.2	Definition of Adaptive Security	19
5	Threshold Schnorr Signature Scheme Sparkle+	19
5.1	Communication Model	20
5.2	Protocol Description	20
6	Our Results	23
6.1	Static Security Under Standard Assumptions	23
6.2	Adaptive Security up to $t/2$ Corruptions	24
6.3	Adaptive Security up to t Corruptions	26
7	Conclusion	27
A	Proof of Static Security	32
B	Proof of Adaptive Security up to $t/2$ Corruptions	41
C	Proof of Adaptive Security up to t Corruptions	50

1 Introduction

A threshold signature scheme allows a set of n possible signers to jointly produce a signature on a message such that it verifies under a single public key, so long as at least a threshold $t + 1$ of signers participate. Importantly, a threshold signature scheme should remain unforgeable even if t signers are under adversarial control. A recent line of work has explored multi-party signatures whose output is a standard (single-party) Schnorr signature [58, 4, 52, 35, 54, 10, 56, 62, 29]. Schnorr signatures admit an efficient and compact representation even in the multi-party setting, which makes them of particular interest for practical use [30, 21].

Static vs. Adaptive Security. Most threshold signature schemes in the literature are proven secure under static corruptions. In the static setting, an adversary must declare which parties it wishes to corrupt in advance of any messages being sent. This model places an artificial restriction on the adversary’s capabilities: in reality, malicious actors may observe a system before targeting specific parties. Thus, adaptive security is a strictly stronger notion, and indeed there are schemes that are statically but not adaptively secure [24]. While there are generic methods for transforming a statically secure scheme into an adaptively secure one [26], such as guessing the corrupted parties and aborting if incorrect, these methods incur undesirable performance overhead and a tightness loss of $\binom{n}{t}$. This is acceptable only for small numbers of parties, and adaptive security without $\binom{n}{t}$ tightness loss is challenging to achieve. A number of other techniques for proving adaptive security have been proposed but require undesirable tradeoffs, such as secure erasure of secret state [26, 41, 42] or a substantial amount of communication [49, 55, 2].

In this work, we investigate the adaptive security of a simple three-round threshold Schnorr signature scheme, which we call **Sparkle+**, under different corruption models and security assumptions. Achieving adaptive security is not just of theoretical interest: NIST recently published a call for multi-party threshold schemes and included adaptive security as a main goal [21, 22], ideally supporting up to $n = 1024$ or more parties. Our techniques are likely of independent interest, as this paper introduces the first proof for *any* threshold Schnorr signature scheme that allows a full t adaptive corruptions out of $t + 1$ signers, for any $t < n$, without secure erasures or $\binom{n}{t}$ tightness loss. Moreover, the reduction for full adaptive security is tight.

Concurrent Security. A *concurrent* adversary may open an arbitrary number of signing sessions simultaneously. Unforgeability against a concurrent adversary is also a difficult property to achieve, and indeed a host of threshold, blind, and multi-signature schemes were demonstrated to be broken by concurrent (ROS) attacks first observed by Drijvers et al. [37] and exhibited in polynomial time by Benhamouda et al. [17]. Our security reductions for **Sparkle+** hold against a concurrent and adaptive adversary. Moreover, **Sparkle+** allows one round of signing to be pre-processed, before the message and signing set are determined. This is optimal, as further pre-processing would make the scheme susceptible to

Scheme	Offline Signing Rounds	Online Signing Rounds	Static Security Assumptions	Proof of Adaptive Security
Lindell22 [54]	-	3	Schnorr with aborts	$\times$
Classic S. [56]	-	3	Schnorr with aborts	$\binom{n}{t}$ security loss
Zero S. [56]	-	3	Schnorr with aborts	Requires erasures
FROST/2/3 [52, 35, 10, 62]	1	1	AOMDL	$\times$
Sparkle+	1	2	DL	$\checkmark$

Fig. 1. Two- and three-round threshold Schnorr signature schemes that are concurrently secure in the random oracle model (ROM). Schnorr with aborts denotes simulation-based security with respect to Schnorr as the idealized functionality, with an adversary that can cause the protocol to abort.

ROS attacks. Combining concurrency and adaptivity in a multi-party, multi-round signing protocol is the main technical achievement of this work.

Static Security of Sparkle+. Sparkle+ is a three-round threshold Schnorr signature scheme that follows a commit-reveal paradigm. The protocol consists of an offline phase, which may be executed before the message or signing set is known, and an online phase consisting of two signing rounds. To begin with, we prove the static security of Sparkle+ from minimal assumptions: that the discrete logarithm assumption (DL) holds in the random oracle model (ROM). This is the same assumption and model for which Schnorr signatures themselves are proven secure [60]. We compare Sparkle+ with existing threshold Schnorr signature schemes in Tables 1 and 3.

Adaptive Security of Sparkle+. We next consider an adaptive adversary who may corrupt up to $t/2$ out of a threshold of $t + 1$ parties over the course of the protocol. We prove that Sparkle+ is adaptively secure in the random oracle model under AOMDL, a weaker variant of the one-more discrete logarithm assumption formalized in [58]. In the $t + 1$ -aomdl game, the adversary is given as input an AOMDL challenge that is a vector of group elements of length $t + 1$. It is also given access to a discrete logarithm oracle, which returns the discrete logarithm of a group element chosen by the adversary. To win the $t + 1$ -aomdl game, the adversary must output all $t + 1$ discrete logarithms of its challenge, having queried its DL oracle a maximum of t times. The AOMDL assumption is stronger than the discrete logarithm assumption because of the adversary’s ability to request for up to t discrete logarithm solutions before returning the $t + 1$ discrete logarithms of its challenge. On the other hand, the AOMDL assumption is strictly weaker than standard OMDL [11] because the adversary only queries for the discrete logarithm of linear combinations of its challenge elements, which means the DL

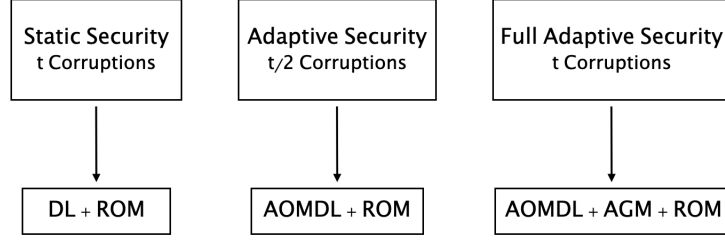


Fig. 2. Comparison of security models and assumptions for our threshold Schnorr signature scheme **Sparkle+**. The signing threshold is $t + 1$. DL is the discrete logarithm assumption, and AOMDL is the algebraic one-more discrete logarithm assumption. ROM is the random oracle model, and AGM is the algebraic group model.

oracle runs in polynomial time. This, in conjunction with the verification running in polynomial time, makes AOMDL a falsifiable assumption [9, 47, 51].

In the case where the adversary can corrupt up to a full t out of $t + 1$ parties, it is not clear how to prove the adaptive security of **Sparkle+** under AOMDL+ROM alone. The reason is that in order to extract an AOMDL solution, the adversary is rewound once, and there is no guarantee that the adversary will corrupt the same set of parties after the fork as it did during the first iteration of the protocol. When the adversary can corrupt only $t/2$ parties, this causes no issues, as the total number of corruptions over both iterations does not exceed t . If the adversary could corrupt more parties, the reduction would query its DL oracle more than t times and would lose the $t + 1$ -aomdl game.

We thus look towards the algebraic group model (AGM) [43] for proving our strongest adaptivity result. The AGM assumes that whenever an adversary outputs a group element, it also outputs an algebraic representation specifying how the group element depends on previously seen values. Schnorr signatures were proven secure in the AGM in [44] to avoid the forking used in the original proof [60]. We use both techniques, to prove the adaptive security of **Sparkle+** against $t/2$ corruptions in the ROM only, by rewinding, and to prove full adaptive security in the AGM and ROM. By full adaptive security, we mean that the adversary can adaptively corrupt up to the maximum of t out of $t + 1$ parties and still cannot forge. Our full adaptive security reduction is *straight-line*, i.e., does not rewind the adversary, and so avoids counting the number of corruptions over different forks of the adversary’s execution.

Our Contributions. The contributions of this work are as follows:

- We introduce **Sparkle+**, an efficient and practical threshold Schnorr signature scheme with one round of pre-processing and two online signing rounds.
- We begin by proving that **Sparkle+** is statically secure under the DL assumption in the ROM.

- We then demonstrate the adaptive security of **Sparkle+** against up to $t/2$ corruptions under the AOMDL assumption in the ROM.
- Finally, we prove our main result: that **Sparkle+** achieves full adaptive security, i.e., up to a full t corruptions, under the AOMDL assumption in the AGM and ROM.

2 Related Work

Threshold Schnorr Signatures. Closest to the design of **Sparkle+** is the MSDL scheme presented by Boneh et al. [19], the three-round MuSig scheme by Maxwell et al. [57], and the 2Schnorr scheme by Nicolosi et al. [59]. However, MSDL and MuSig consider only the multi-signature setting (n -out-of- n), and 2Schnorr considers only the 2-out-of-2 setting. For multi-signatures, security holds against t corrupt parties out of $t + 1$ parties.

We now review threshold Schnorr schemes that are secure in the concurrent setting.

Stinson and Stroh [65] propose a threshold Schnorr signature scheme secure in the random oracle model under the discrete logarithm assumption. However, their scheme requires performing a three-round distributed key generation protocol (DKG) [46] to generate the nonce for each signature, which adds considerable network overhead: at a minimum, it requires participants to perform four rounds in total. Furthermore, the proof of security assumes only a static adversary.

Abe and Fehr [2] achieve a fully adaptively secure threshold Schnorr signature scheme also by using a three-round DKG for nonce generation. Security is modeled in the universally composable framework (UC) [23] and therefore captures a concurrent adversary. However, their scheme requires strong network assumptions (synchronous broadcast) and only achieves adaptive security in the honest majority setting ($t < n/2$). In contrast, **Sparkle+** is secure in the asynchronous setting, does not assume broadcast or even authenticated channels, and is fully adaptively secure for any corruption threshold $t < n$.

Komlo and Goldberg [52] present a two-round threshold Schnorr signature FROST. Unlike prior threshold Schnorr schemes in the literature [45], FROST is secure against a concurrent adversary and is not susceptible to ROS attacks [17]. FROST2, proposed by Crites et al. [35] and refined by Bellare et al. [16, 10], is an optimized version of FROST that reduces the number of exponentiations required for signing from $t + 1$ to one. A further optimization of FROST2, called FROST3, is introduced by Ruffing et al. [62] and proven secure with a DKG by Chu et al. [29]. (See Table 3 for a comparison of efficiency.) However, all three of FROST, FROST2, FROST3 are only proven statically secure, under the algebraic one-more discrete logarithm assumption (AOMDL) [10]. While **Sparkle+** adds an additional round of communication, it requires only a single exponentiation per signer and is proven statically secure under standard assumptions, which are also criteria of interest in [21, 22].

Lindell presents a three-round threshold Schnorr signature [54] with the goal of defining a scheme that is UC-secure in the random oracle model under the discrete

Scheme	Sign					Combine	
	Performance			Bandwidth		Performance	
	rounds	exp	H	$\mathbb{G}$	$\mathbb{F}$	exp	H
FROST [52]	2	$t + 2$	$t + 1$	2	1	t	t
FROST2 [10]	2	3	2	2	1	1	1
FROST3 [62]	2	3	2	1	1	1	1
Classic S. [56]	3	1	1	1	2	0	0
Lindell22 [54]	3	$11t + 1$	$18t + 10^7$	11	25	0	0
Sparkle+	3	1	$t + 2$	1	2	0	0

Fig. 3. Efficiency of Two- and Three-Round Threshold Schnorr Signature Schemes. All output a standard Schnorr signature. We only compare schemes that are secure against ROS attacks [37, 17]. The number of network rounds between participants is given in the rounds column. exp stands for the number of group exponentiations. The total number of group and field elements sent by each signer is denoted by $\mathbb{G}$ and $\mathbb{F}$, respectively. H denotes the total number of hashing operations performed. The cost of signature verification is identical for each scheme, and is simply the cost of verifying a single Schnorr signature. Estimates for Lindell22 are made with respect to a 128-bit security level for Fischlin [40], where $r = 8$ is the number of commitments for a Fischlin proof and the length of the zero vector is $b = 16$, such that $b \cdot r = 128$.

logarithm assumption only. However, the security proof assumes the adversary is static. **Sparkle+** similarly relies on only the ROM and DL assumption for static security, but does not require the use of online-extractable zero-knowledge proofs [40], and hence is both significantly more efficient and a simpler design.

Concurrent to this work, Makriyannis [56] defines a commit-reveal threshold Schnorr signature scheme similar to **Sparkle+** (though without pre-processing), called Classic S., and proves security with respect to an idealized threshold signature functionality by Canetti et al. [25]. They consider adaptive security but employ the guessing argument that incurs $\binom{n}{t}$ tightness loss.

Adaptive Security of Threshold Signatures. While adaptive security for threshold schemes is a well-known topic in the literature, no proof of full adaptive security for any $t \leq n$ without $\binom{n}{t}$ tightness loss or secure erasures exists for a threshold Schnorr signature scheme in the concurrent setting. We now review several alternative signature schemes and techniques for achieving adaptive security.

Canetti et al. [26] and Frankel et al. [41, 42] independently introduce the “Single Inconsistent Player” (SIP) technique for transforming statically secure threshold schemes into adaptively secure schemes. Subsequent works [49, 55, 2] use these techniques to prove adaptive security under concurrent composition and without erasures. First, a secret key x is n -out-of- n shared between all parties $x = x_1 + \dots + x_n$. Then, each x_i is further t -out-of- n shared. If a party does not participate in signing, then other parties can reconstruct their contribution.

In the simulation, a single party’s secret x_i is not defined, and the simulation aborts if the i^{th} party is corrupted. The SIP technique elegantly ensures that the simulation only aborts with constant probability such that a polynomial-time reduction can be repeated until successful. In contrast, our simulator can respond to corruption queries for any party.

Almansa et al. [3] present a threshold RSA scheme with proactive and adaptive security, but these results do not translate to the discrete logarithm setting. Libert et al. [53] propose a variant of the threshold BLS [18] scheme that is adaptively secure. However, it is incompatible with single-party BLS verification, an often-critical goal for threshold schemes in practice. Bacho and Loss [6] demonstrate the adaptive security of threshold BLS in Type I bilinear groups directly in the AGM from the $t + 1$ -omdl assumption. Their reduction is tight. Interestingly, they demonstrate that the $t + 1$ -omdl assumption is the minimum assumption under which threshold BLS can be proven adaptively secure.

We similarly rely on the $t + 1$ -(a)omdl assumption for the adaptive security of Sparkle+. In more detail, the $t + 1$ -(a)omdl assumption gives the reduction t -time access to an oracle that returns the discrete logarithm of a group element of its choosing. This oracle is used in both [6] and our reduction to obtain the *secret keys* of parties as they are corrupted, by querying on the public keys of the chosen parties. Since threshold BLS [6] is non-interactive, secret keys are the extent of the private information gained by the adversary through adaptive corruption. However, for interactive protocols, the reduction must not only simulate keys, but any *state* used during signing, and it must correctly simulate the corruption oracle for each round in the signing protocol (without erasing state, to avoid the assumption of secure erasures). Our technique for simulating state across three signing rounds and potentially many concurrent signing sessions, without relying on erasure of state, is the main technical achievement of this work.

Subsequent Work to Our Original Work [34]. Since our original work, there has been a surge of research on adaptive security of threshold schemes.

Das and Ren [36] show adaptive security of threshold BLS in Type III bilinear groups under the co-computational Diffie-Hellman assumption (co-CDH) and decisional Diffie-Hellman assumption (DDH) in the ROM.

Makriyannis [56] introduces a threshold Schnorr signature scheme, Zero S., which relies on Pedersen commitments, Fischlin proofs of knowledge [40], and an interactive protocol to identify misbehaving parties. Zero S. is proven adaptively secure in a straight-line manner with a reduction to DL but requires secure erasure of secret state.

Twinkle, by Bacho et al. [8], is a family of three-round threshold signature schemes. The first, efficient construction is a distributed Chaum-Pedersen proof of equality of discrete logarithms $X = g^x$ and $H(m)^x$ [27] (vs. a proof of knowledge of the discrete logarithm of $X = g^x$, as in Schnorr signatures). Thus, signatures are Chaum-Pedersen outputs $(H(m)^x, c, z)$, where (c, z) is a Schnorr signature. This scheme is proven fully adaptively secure under a one-more variant of the computational Diffie-Hellman assumption (OMCDH) in the ROM. The second, matrix construction is proven fully adaptively secure under the DDH assumption

in the ROM. The adaptive simulation does not allow pre-processing: the message, which is hashed in the first round, is given along with the signing set in the first round. Furthermore, neither scheme is compatible with Schnorr signatures.

Katsumata et al. [50] prove the full adaptive security of two threshold Schnorr schemes, Crackle and Snap, using rewinding and assuming the EUF-CMA security of an additional signature scheme. Adaptive security critically relies on partial public keys not being revealed, however, and neither scheme allows for identifiable abort. That is, there is no way to identify even a single malicious party when the signing protocol results in an invalid signature. Moreover, the schemes require five and four online signing rounds, respectively.

Chen [28] proves the full adaptive security of a three-round threshold Schnorr scheme based on FROST, called FROST-Mask, and a two-round scheme that requires (consistent) session identifiers, called HBTS-Mask. As techniques from [50] are used, both schemes inherit the lack of identifiable abort.

Glacius, by Bacho et al. [5], is a five-round threshold Schnorr signature scheme that is proven fully adaptively secure under DDH in the ROM using similar techniques to [36]. Its security relies on authenticated channels between all participants.

HARTS, by Bacho et al. [7], is a robust, constant-round threshold Schnorr protocol that achieves adaptive security in the $t/2$ setting under AOMDL in the ROM, assuming secure erasure of secret state and authenticated channels.

Crites et al. [32] prove a series of impossibility results for threshold signature schemes with keys of a common form, which they call key-unique schemes. In particular, they show that key-unique threshold Schnorr signature schemes cannot be proven adaptively secure above $t/2$ corruptions under (A)OMDL in the programmable ROM for a fully black-box reduction that rewinds the adversary and programs with fresh randomness on at least one random oracle query. This is exactly the type of reduction we employ in our $t/2$ adaptive security proof, confirming their result of a maximum corruption threshold of $t/2$.

Definitions. In this work, we employ a game-based approach to defining the static and adaptive security of a threshold signature, formalizing prior notions presented in the literature [53]. Alternative definitions of adaptive security in the UC setting have been proposed by Canetti et al. [25]. They prove their threshold ECDSA scheme adaptively secure assuming that ECDSA is secure, in addition to other non-interactive and falsifiable assumptions. However, their construction focuses on n -out-of- n multi-party signing, and their techniques critically do not translate to the $(t+1)$ -out-of- n setting unless $\binom{n}{t}$ is small. Our fully adaptive $(t+1)$ -out-of- n construction requires the algebraic group model, and incorporating algebraic adversaries into the UC setting is known to be a hard problem [1].

On the other hand, Lindell [54] shows that their protocol UC-realizes the Schnorr functionality with aborts, in the presence of an adversary that non-adaptively corrupts t parties. Secure evaluation of the Schnorr functionality is stronger than unforgeability. As noted in [25], it is arguably overly strong in the sense that it necessitates certain design decisions, such as incorporating online-extractable zero-knowledge proofs. Indeed, [54] elected for Fischlin proofs

(see Table 3). The only method to bias the nonces in both **Sparkle+** and [54] is to abort; this is not the case in FROST [52]. However, all three works allow aborts, and therefore the distribution of the secret randomness cannot be considered uniform [38].

Differences to the Original Work [34]. The authors of [8] identified one class of adversarial behavior not captured by our original security analysis, namely that honest parties may not have the same view of commitments generated in the first signing round. This is fixed by having parties sign the view of the commitments they receive from other parties, using any EUF-CMA-secure signature scheme.⁴ In particular, a Schnorr signature may be used, which we assume in Fig. 2 and Tables 1 and 3 since it is itself secure under the DL assumption in the ROM [60]. Signature-based authentication is used in other recent works, such as a version of FROST achieving a higher level of security (TS-UF-4) in the formalism of Bellare et al. [10], in the blind threshold signature scheme Snowblind [30], for identifiable abort in the five-round scheme Glacius [7], and for achieving adaptive security for the five- and four-round schemes Crackle and Snap [50]. In this work, we are able to achieve adaptive security for a Schnorr threshold signature scheme with only two online signing rounds (where the message and signing set deferred to the second round). We prove adaptive security of **Sparkle+**, with pre-processing, using our original simulation techniques.

3 Preliminaries

3.1 General Notation and Definitions

Let $\kappa \in \mathbb{N}$ denote the security parameter and 1^κ its unary representation. A function $\nu : \mathbb{N} \rightarrow \mathbb{R}$ is called *negligible* if for all $c \in \mathbb{R}, c > 0$, there exists $k_0 \in \mathbb{N}$ such that $|\nu(k)| < \frac{1}{k^c}$ for all $k \in \mathbb{N}, k \geq k_0$. For a non-empty set S , let $x \leftarrow^s S$ denote sampling an element of S uniformly at random and assigning it to x . We use $[n]$ to represent the set $\{1, \dots, n\}$ and $[0..n]$ to represent the set $\{0, \dots, n\}$. We represent vectors as $\vec{a} = (a_1, \dots, a_n)$.

Let PPT denote probabilistic polynomial time. Algorithms are randomized unless explicitly noted otherwise. Let $y \leftarrow A(x; \rho)$ denote running algorithm A on input x and randomness ρ and assigning its output to y . Let $y \leftarrow^s A(x)$ denote $y \leftarrow A(x; \rho)$ for a uniformly random ρ . The set of values that have non-zero probability of being output by A on input x is denoted by $[A(x)]$.

Group Generation. Let GrGen be a polynomial-time algorithm that takes as input a security parameter 1^κ and outputs a group description $(\mathbb{G}, p, g)$ consisting of a group $\mathbb{G}$ of order p , where p is a κ -bit prime, and a generator g of $\mathbb{G}$.

⁴ We place minimal requirements on the communication network for signing to give practitioners the flexibility to implement Sparkle+ in myriad settings, without the risk of compromising security. See Section 5.1 for more details on our communication model.

Random Oracle Model [14]. The random oracle model is an idealized model that treats a hash function H as an oracle in the following way. When queried on an input in the domain of H , the oracle first checks if it has an entry stored in its table for this input. If so, it returns this value. If not, it samples an output in the codomain of H uniformly at random, stores the input-output pair in its table, and returns the output.

Algebraic Group Model [43]. The algebraic group model is an idealized model that places the following restriction on the computation of the adversary. An adversary is *algebraic* if for every group element $Z \in \mathbb{G} = \langle g \rangle$ that it outputs, it is required to output a representation $\vec{a} = (a_0, a_1, a_2, \dots)$ such that $Z = g^{a_0} \prod Y_i^{a_i}$, where $Y_1, Y_2, \dots \in \mathbb{G}$ are group elements that the adversary has seen thus far. Intuitively, this captures the notion that an algorithm should know how it computes its outputs from the values it has received as input so far. The AGM is reminiscent of the generic group model (GGM), but lies somewhere between it and the standard model.

Polynomial Interpolation. A polynomial $f(x) = a_0 + a_1x + a_2x^2 + \dots + a_tx^t$ of degree t over a field $\mathbb{F}$ can be interpolated by $t + 1$ points. Let $\eta \subseteq [n]$ be the list of $t + 1$ distinct indices corresponding to the x -coordinates $x_i \in \mathbb{F}, i \in \eta$, of these points. Then the Lagrange polynomial $L_i(x)$ has the form:

$$L_i(x) = \prod_{j \in \eta; j \neq i} \frac{x - x_j}{x_i - x_j} \quad (1)$$

Given a set of $t + 1$ points $(x_i, f(x_i))_{i \in [t+1]}$, any point $f(x_\ell)$ on the polynomial f can be determined by Lagrange interpolation as follows:

$$f(x_\ell) = \sum_{k \in \eta} f(x_k) \cdot L_k(x_\ell)$$

Definition 1 (Shamir Secret Sharing [64]). *Shamir secret sharing is an $(n, t + 1)$ -threshold secret sharing scheme consisting of algorithms (IssueShares, Recover), defined as follows:*

- **IssueShares** $(x, n, t + 1) \rightarrow \{(1, x_1), \dots, (n, x_n)\}$: On input a secret x , number of participants n , and threshold $t + 1$, perform the following. First, define a polynomial $f(Z) = x + a_1Z + a_2Z^2 + \dots + a_tZ^t$ by sampling t coefficients at random: $a_1, \dots, a_t \leftarrow^s \mathbb{Z}_p$. Then, set each participant's share $x_i, i \in [n]$, to be the evaluation of $f(i)$:

$$x_i \leftarrow x + \sum_{j \in [t]} a_j i^j$$

Output $\{(i, x_i)\}_{i \in [n]}$.

- **Recover** $(t + 1, \{(i, x_i)\}_{i \in \mathcal{S}}) \rightarrow \perp/x$: On input threshold $t + 1$ and a set of shares $\{(i, x_i)\}_{i \in \mathcal{S}}$, output $\perp$ if $\mathcal{S} \not\subseteq [n]$ or if $|\mathcal{S}| < t + 1$. Otherwise, recover x as follows:

$$x \leftarrow \sum_{i \in \mathcal{S}} \lambda_i x_i$$

where the Lagrange coefficient for the set $\mathcal{S}$ is defined by

$$\lambda_i = \prod_{j \in \mathcal{S}, j \neq i} \frac{j}{j-i}$$

3.2 Digital Signatures

Definition 2 (Digital Signatures). A digital signature scheme DS consists of polynomial-time algorithms $(\text{DS.Setup}, \text{DS.KeyGen}, \text{DS.Sign}, \text{DS.Verify})$, defined as follows:

- $\text{DS.Setup}(1^\kappa) \rightarrow \text{par}_{\text{DS}}$: On input a security parameter 1^κ , this algorithm outputs public parameters par_{DS} (which are given implicitly as input to all other algorithms).
- $\text{DS.KeyGen}() \rightarrow (\hat{\text{pk}}, \hat{\text{sk}})$: This probabilistic algorithm outputs a public key $\hat{\text{pk}}$ and secret key $\hat{\text{sk}}$.
- $\text{DS.Sign}(\hat{\text{sk}}, \text{msg}) \rightarrow \hat{\sigma}$: On input a secret key $\hat{\text{sk}}$ and a message msg , this algorithm outputs a signature $\hat{\sigma}$.
- $\text{DS.Verify}(\hat{\text{pk}}, \text{msg}, \hat{\sigma}) \rightarrow 0/1$: On input a public key $\hat{\text{pk}}$, a message msg , and a purported signature $\hat{\sigma}$, this deterministic algorithm outputs 1 (accept) if $\hat{\sigma}$ is a valid signature on msg under $\hat{\text{pk}}$; else, it outputs 0 (reject).

A digital signature scheme must be *correct*. It is said to be *secure* if it is existentially unforgeable under chosen-message attacks (EUF-CMA).

Definition 3 (Correctness). A digital signature scheme $\text{DS} = (\text{DS.Setup}, \text{DS.KeyGen}, \text{DS.Sign}, \text{DS.Verify})$ is correct if for all security parameters $\kappa \in \mathbb{N}$, all key pairs $(\hat{\text{pk}}, \hat{\text{sk}}) \in [\text{DS.KeyGen}()]$, and all messages msg , we have:

$$\Pr[\text{DS.Verify}(\hat{\text{pk}}, \text{msg}, \text{DS.Sign}(\hat{\text{sk}}, \text{msg})) = 1] = 1$$

Definition 4 (EUF-CMA [48]). Let the advantage of an adversary $\mathcal{A}$ playing the EUF-CMA game, $\text{Game}_{\mathcal{A}, \text{DS}}^{\text{EUF-CMA}}(\kappa)$, as defined in Figure 4, be as follows:

$$\text{Adv}_{\mathcal{A}, \text{DS}}^{\text{EUF-CMA}}(\kappa) = \Pr[\text{Game}_{\mathcal{A}, \text{DS}}^{\text{EUF-CMA}}(\kappa) = 1]$$

A digital signature scheme $\text{DS} = (\text{DS.Setup}, \text{DS.KeyGen}, \text{DS.Sign}, \text{DS.Verify})$ is existentially unforgeable under chosen-message attacks if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{DS}}^{\text{EUF-CMA}}(\kappa)$ is negligible.

Definition 5 (Schnorr Signatures [63]). The Schnorr signature scheme consists of polynomial-time algorithms $(\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$, defined as follows:

MAIN $\text{Game}_{\mathcal{A}, \text{DS}}^{\text{EUF-CMA}}(\kappa)$	$\mathcal{O}^{\text{DS.Sign}}(\text{msg})$
$\text{par}_{\text{DS}} \leftarrow \text{DS.Setup}(1^\kappa)$	$\mathcal{Q}_{\text{msg}} \leftarrow \mathcal{Q}_{\text{msg}} \cup \{\text{msg}\}$
$\mathcal{Q}_{\text{msg}} \leftarrow \emptyset$	$\hat{\sigma} \leftarrow \text{DS.Sign}(\hat{\text{sk}}, \text{msg})$
$(\hat{\text{pk}}, \hat{\text{sk}}) \leftarrow \text{DS.KeyGen}()$	return $\hat{\sigma}$
$(\text{msg}^*, \hat{\sigma}^*) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{DS.Sign}}}(\hat{\text{pk}})$	
return 0 if $\text{msg}^* \in \mathcal{Q}_{\text{msg}}$	
$\vee \text{DS.Verify}(\hat{\text{pk}}, \text{msg}^*, \hat{\sigma}^*) \neq 1$	
return 1	

Fig. 4. The EUF-CMA security game for a digital signature scheme DS. The public parameters par_{DS} are implicitly given as input to all algorithms.

- $\text{Setup}(1^\kappa) \rightarrow \text{par}$: On input a security parameter 1^κ , run $(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\kappa)$ and select a hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$. Output public parameters $\text{par} \leftarrow ((\mathbb{G}, p, g), H)$ (which are given implicitly as input to all other algorithms).
- $\text{KeyGen}() \rightarrow (\text{pk}, \text{sk})$: Sample a secret key $x \leftarrow \mathbb{Z}_p$ and compute the public key as $X \leftarrow g^x$. Output key pair $(\text{pk}, \text{sk}) \leftarrow (X, x)$.
- $\text{Sign}(\text{sk}, m) \rightarrow \sigma$: On input a secret key $\text{sk} = x$ and a message m , sample a nonce $r \leftarrow \mathbb{Z}_p$. Then, compute a nonce commitment $R \leftarrow g^r$, the challenge $c \leftarrow H(X, m, R)$, and the response $z \leftarrow r + cx$. Output a signature $\sigma \leftarrow (R, z)$.
- $\text{Verify}(\text{pk}, m, \sigma) \rightarrow 0/1$: On input a public key $\text{pk} = X$, a message m , and a purported signature $\sigma = (R, z)$, compute $c \leftarrow H(X, m, R)$ and output 1 (accept) if $R \cdot X^c = g^z$; else, output 0 (reject).

A Schnorr signature is a Sigma protocol zero-knowledge proof of knowledge (as in Boneh–Shoup, §19.4 [20]) of the discrete logarithm of the public key X , made non-interactive and bound to the message m by the Fiat-Shamir transform [39]. Schnorr signatures are secure under the discrete logarithm assumption in the random oracle model [60].

3.3 Assumptions

Assumption 1 (Discrete Logarithm Assumption (DL)) Let GrGen be a group generator that outputs $(\mathbb{G}, p, g)$. Let the advantage of an adversary $\mathcal{A}$ playing the discrete logarithm game, $\text{Game}_{\mathcal{A}}^{\text{dl}}(\kappa)$, as defined in Figure 5, be as follows:

$$\text{Adv}_{\mathcal{A}}^{\text{dl}}(\kappa) = \Pr[\text{Game}_{\mathcal{A}}^{\text{dl}}(\kappa) = 1]$$

The discrete logarithm assumption holds relative to GrGen if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{dl}}(\kappa)$ is negligible.

MAIN $\text{Game}_{\mathcal{A}}^{\text{dl}}(\kappa)$	MAIN $\text{Game}_{\mathcal{A}}^{t+1-\text{a-omdl}}(\kappa)$	$\mathcal{O}^{\text{dl}}(X, \alpha, \{\beta_i\}_{i=0}^t)$
$(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\kappa)$	$(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\kappa)$	$\parallel X = g^\alpha \prod_{i=0}^t X_i^{\beta_i}$
$x \leftarrow \mathbb{Z}_p; X \leftarrow g^x$	$q \leftarrow 0$	return $\perp$ if $q = t$
$x' \leftarrow \mathcal{A}((\mathbb{G}, p, g), X)$	for $i \in [0..t]$ do	$q \leftarrow q + 1$
if $x' = x$	$x_i \leftarrow \mathbb{Z}_p; X_i \leftarrow g^{x_i}$	$x \leftarrow \alpha + \sum_{i=0}^t x_i \beta_i$
return 1	$\vec{x} \leftarrow (x_0, \dots, x_t)$	return x
return 0	$\vec{X} \leftarrow (X_0, X_1, \dots, X_t)$	$x \leftarrow \text{dlog}(X)$
	$\vec{x}' \leftarrow \mathcal{A}^{\mathcal{O}^{\text{dl}}}((\mathbb{G}, p, g), \vec{X})$	return x
	if $\vec{x}' = \vec{x}$	
	return 1	
	return 0	

Fig. 5. The Discrete Logarithm (DL), One-More Discrete Logarithm (OMDL), and Algebraic One-More Discrete Logarithm (AOMDL) games. The differences between the OMDL and AOMDL games are highlighted in gray. The key distinction is that in the AOMDL game, the adversary can only query its discrete logarithm oracle on linear combinations of its challenge group elements; in the OMDL game, the environment must return the discrete logarithm of an arbitrary group element. **dlog** is an algorithm that finds the discrete logarithm of an arbitrary group element X base g .

Assumption 2 (One-More Discrete Logarithm Assumption (OMDL))
[11] Let GrGen be a group generator that outputs $(\mathbb{G}, p, g)$. Let the advantage of an adversary $\mathcal{A}$ playing the $t+1$ -one-more discrete logarithm game, $\text{Game}_{\mathcal{A}}^{t+1-\text{omdl}}(\kappa)$, as defined in Figure 5, be as follows:

$$\text{Adv}_{\mathcal{A}}^{t+1-\text{omdl}}(\kappa) = \Pr[\text{Game}_{\mathcal{A}}^{t+1-\text{omdl}}(\kappa) = 1]$$

The one-more discrete logarithm assumption holds relative to GrGen if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{t+1-\text{omdl}}(\kappa)$ is negligible.

Algebraic One-More Discrete Logarithm Assumption (AOMDL). The algebraic one-more discrete logarithm assumption (AOMDL) was introduced formally by Jonas, Ruffing, and Seurin for proving the security of the two-round Schnorr multi-signature scheme MuSig2 [58]. We use it as an assumption to prove the adaptive security of **Sparkle+**, but it actually (implicitly) appears elsewhere in the literature where OMDL is used [13, 59, 15, 44].

As in the standard OMDL game, the AOMDL adversary is given as input a group description $(\mathbb{G}, p, g)$ and a set of challenge group elements $(X_0, X_1, \dots, X_t)$ and has access to a discrete logarithm solution oracle $\mathcal{O}^{\text{dl}}$ that it may query up to t times. The adversary wins the game if it computes the discrete logarithms $(x_0, x_1, \dots, x_t)$ without exceeding t queries to $\mathcal{O}^{\text{dl}}$ (i.e., the adversary is able to

compute “one more” discrete logarithm). The difference in the AOMDL game is that, when the adversary queries the $\mathcal{O}^{\text{dl}}$ oracle on a group element X , it must provide an algebraic representation $(\alpha, \{\beta_i\}_{i=0}^t)$ of X relative to the generator g and challenge group elements $(X_0, X_1, \dots, X_t)$, so that $X = g^\alpha \prod_{i=0}^t X_i^{\beta_i}$. The $\mathcal{O}^{\text{dl}}$ oracle may then use this representation to compute $\alpha + \sum_{i=0}^t \beta_i x_i$. This is in contrast to standard OMDL, where the $\mathcal{O}^{\text{dl}}$ oracle may be queried for discrete logarithms of arbitrary group elements. Thus, AOMDL is a weaker assumption than standard OMDL, because it is falsifiable. Indeed, not only does the algorithm verifying the correctness of the AOMDL solutions run in polynomial time, but the $\mathcal{O}^{\text{dl}}$ algorithm does as well.

Assumption 3 (Algebraic One-More Discrete Logarithm Assumption)
[58] *Let GrGen be a group generator that outputs $(\mathbb{G}, p, g)$. Let the advantage of an adversary $\mathcal{A}$ playing the $t+1$ -algebraic one-more discrete logarithm game, $\text{Game}_{\mathcal{A}}^{t+1-\text{aomdl}}(\kappa)$, as defined in Figure 5, be as follows:*

$$\text{Adv}_{\mathcal{A}}^{t+1-\text{aomdl}}(\kappa) = \Pr[\text{Game}_{\mathcal{A}}^{t+1-\text{aomdl}}(\kappa) = 1]$$

The algebraic one-more discrete logarithm assumption holds relative to GrGen if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{t+1-\text{aomdl}}(\kappa)$ is negligible.

4 Threshold Signature Schemes

We begin with the definition of a threshold signature scheme. Our syntax assumes a centralized key generation algorithm to generate the threshold public key pk and key pairs $\{(\text{pk}_i, \text{sk}_i)\}_{i \in [n]}$ for all n parties. However, our construction and proofs can be adapted to use a distributed key generation protocol (DKG). We refer to [6] for a discussion on how to achieve adaptively secure DKGs. Outputting public key shares is also optional.

Our definition models a three-round signing protocol, where the message and signing set are given in the first round. We then provide game-based definitions of static and adaptive security for threshold signature schemes with pre-processing (i.e., where the message and signing set are deferred to the second round), as these are the notions of security achieved by Sparkle+.

Definition 6 (Threshold Signatures). *A threshold signature scheme TS with three signing rounds consists of polynomial-time algorithms $\text{TS} = (\text{Setup}, \text{KeyGen}, (\text{Sign}_1, \text{Sign}_2, \text{Sign}_3), \text{Combine}, \text{Verify})$, defined as follows:*

- $\text{Setup}(1^\kappa) \rightarrow \text{par}$: *This algorithm takes as input a security parameter 1^κ and outputs public parameters par (which are given implicitly as input to all other algorithms).*
- $\text{KeyGen}(n, t+1) \rightarrow (\text{pk}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [n]})$: *A probabilistic algorithm that takes as input the number of signers n and the threshold $t+1$ and outputs the public key pk representing the set of n signers, the set $\{\text{pk}_i\}_{i \in [n]}$ of public keys for each signer, and the set $\{\text{sk}_i\}_{i \in [n]}$ of secret keys for each signer. It is assumed that participant i is sent its secret key sk_i privately (see Section 5.1).*

$\text{TSignHon}(\text{pk}, \{\text{sk}_i\}_{i \in [\mathcal{S}]}, \mathcal{S}, m)$
$\text{return } \perp \text{ if } \mathcal{S} \not\subseteq [n] \wedge \mathcal{S} < t + 1$
$\text{for } k \in \mathcal{S} \text{ do } (\text{pm}_{k,1}, \text{st}_{k,1}) \leftarrow \text{Sign}_1(k, \mathcal{S}, m, \text{sk}_k)$
$\text{for } k \in \mathcal{S} \text{ do } (\text{pm}_{k,2}, \text{st}_{k,2}) \leftarrow \text{Sign}_2(k, \text{st}_{k,1}, \{\text{pm}_{i,1}\}_{i \in \mathcal{S}})$
$\text{for } k \in \mathcal{S} \text{ do } \text{pm}_{k,3} \leftarrow \text{Sign}_3(k, \text{st}_{k,2}, \{\text{pm}_{i,2}\}_{i \in \mathcal{S}})$
$\sigma \leftarrow \text{Combine}(\mathcal{S}, m, \{\text{pm}_{i,1}, \text{pm}_{i,2}, \text{pm}_{i,3}\}_{i \in \mathcal{S}})$
$\text{return } \sigma$

Fig. 6. An algorithm TSignHon for honestly generating a threshold signature in a three-round signing protocol.

- $(\text{Sign}_1, \text{Sign}_2, \text{Sign}_3)$: A set of algorithms where each subsequent algorithm represents a single stage in an interactive three-round signing protocol, performed by each signing party in a signing set $\mathcal{S} \subseteq [n], |\mathcal{S}| \geq t + 1$, with respect to a message m , defined as follows:

$$\begin{aligned}
 (\text{pm}_{k,1}, \text{st}_{k,1}) &\leftarrow \text{Sign}_1(k, \mathcal{S}, m, \text{sk}_k) \\
 (\text{pm}_{k,2}, \text{st}_{k,2}) &\leftarrow \text{Sign}_2(k, \text{st}_{k,1}, \{\text{pm}_{i,1}\}_{i \in \mathcal{S}}) \\
 \text{pm}_{k,3} &\leftarrow \text{Sign}_3(k, \text{st}_{k,2}, \{\text{pm}_{i,2}\}_{i \in \mathcal{S}})
 \end{aligned} \tag{2}$$

where $\text{pm}_{k,1}, \text{pm}_{k,2}, \text{pm}_{k,3}$ are protocol messages sent by party k , and $\text{st}_{k,1}, \text{st}_{k,2}$ is the state of party k at the end of each round.

- $\text{Combine}(\mathcal{S}, m, \{\text{pm}_{i,1}, \text{pm}_{i,2}, \text{pm}_{i,3}\}_{i \in \mathcal{S}}) \rightarrow \sigma$: A deterministic algorithm that takes as input a signing set $\mathcal{S}$, a message m , and a set of protocol messages $\{\text{pm}_{i,1}, \text{pm}_{i,2}, \text{pm}_{i,3}\}_{i \in \mathcal{S}}$ and outputs a signature σ .
- $\text{Verify}(\text{pk}, m, \sigma) \rightarrow 0/1$: A deterministic algorithm that takes as input the public key pk , a message m , and purported signature σ and outputs 1 (accept) if σ is a valid signature on m under pk ; else, it outputs 0 (reject).

Remark 1 (Combine). The Combine algorithm may be performed by one of the signers or some external party.

Correctness. A threshold signature scheme is *correct* if for all security parameters

$\kappa \in \mathbb{N}$, all $\text{par} \in [\text{Setup}(1^\kappa)]$, all $(\text{pk}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [n]}) \in [\text{KeyGen}(n, t + 1)]$, all $\mathcal{S} \subseteq [n]$ such that $|\mathcal{S}| \geq t + 1$, and all messages m , we have:

$$\Pr[\text{Verify}(\text{pk}, m, \sigma) = 1 \mid \sigma \leftarrow \text{TSignHon}(\text{pk}, \{\text{sk}_i\}_{i \in [\mathcal{S}]}, \mathcal{S}, m)] = 1$$

where the algorithm TSignHon is as defined in Fig. 6, which models an honest execution of the signing protocol.

$\text{MAIN Game}_{\mathcal{A}, \mathcal{TS}}^{\overline{\text{adp}}\text{-TS-EUF-CMA}}(\kappa, \text{frac})$	$\mathcal{O}^{\text{Sign}_1}(k)$
$\text{par} \leftarrow \text{Setup}(1^\kappa)$	$\text{return } \perp \text{ if } k \notin \text{hon}$
$\mathbf{Q}_{\text{st}} \leftarrow \emptyset, \mathbf{Q}_m \leftarrow \emptyset$	$\text{eid} \leftarrow_{\$} \{0, 1\}^*$
$(n, t+1, \text{cor}, \text{st}_A) \leftarrow_{\$} \mathcal{A}(\text{par})$	$(\text{pm}_{k, \text{eid}, 1}, \text{st}_{k, \text{eid}, 1}) \leftarrow \text{Sign}_1(k, \text{sk}_k)$
$\text{return } \perp \text{ if } t+1 > n \vee \text{cor} \not\subseteq [n]$	$\mathbf{Q}_{\text{st}}[k, \text{eid}, 1] \leftarrow \text{st}_{k, \text{eid}, 1}$
$\vee \text{cor} > \text{frac} \cdot t$	$\text{return } (\text{eid}, \text{pm}_{k, \text{eid}, 1})$
$\text{hon} \leftarrow [n] \setminus \text{cor}$	$\mathcal{O}^{\text{Sign}_2}(k, \text{eid}, \mathcal{S}, m, \{\text{pm}_{i, \text{eid}_i, 1}\}_{i \in \mathcal{S}})$
$(\text{pk}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [n]}) \leftarrow_{\$} \text{KeyGen}(n, t+1)$	$\text{return } \perp \text{ if } k \notin \text{hon}$
$\text{input} \leftarrow (\text{pk}, \{\text{pk}_i\}_{i \in [n]}, \{\text{sk}_j\}_{j \in \text{cor}}, \text{st}_A)$	$\text{return } \perp \text{ if } \mathbf{Q}_{\text{st}}[k, \text{eid}, 1] = \perp$
$(m^*, \sigma^*) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}^{\text{Sign}_1, \text{Sign}_2, \text{Sign}_3, \text{Corrupt}}}}(\text{input})$	$\text{return } \perp \text{ if } \mathbf{Q}_{\text{st}}[k, \text{eid}, 2] \neq \perp$
$\text{return } 1 \text{ if } m^* \notin \mathbf{Q}_m$	$\text{parse } \text{st}_{k, \text{eid}, 1} \leftarrow \mathbf{Q}_{\text{st}}[k, \text{eid}, 1]$
$\wedge \text{Verify}(\text{pk}, m^*, \sigma^*) = 1$	$\mathbf{Q}_m \leftarrow \mathbf{Q}_m \cup \{m\}$
$\text{return } 0$	$\text{return } \perp \text{ if } \perp$
$\mathcal{O}^{\text{Corrupt}}(k)$	$\leftarrow \text{Sign}_2(k, \text{st}_{k, \text{eid}, 1}, \mathcal{S}, m, \{\text{pm}_{i, \text{eid}_i, 1}\}_{i \in \mathcal{S}})$
$\text{return } \perp \text{ if } k \notin \text{hon} \vee \text{cor} \geq \text{frac} \cdot t$	$(\text{pm}_{k, \text{eid}, 2}, \text{st}_{k, \text{eid}, 2})$
$\text{cor} \leftarrow \text{cor} \cup \{k\}$	$\leftarrow \text{Sign}_2(k, \text{st}_{k, \text{eid}, 1}, \mathcal{S}, m, \{\text{pm}_{i, \text{eid}_i, 1}\}_{i \in \mathcal{S}})$
$\text{hon} \leftarrow \text{hon} \setminus \{k\}$	$\mathbf{Q}_{\text{st}}[k, \text{eid}, 2] \leftarrow \text{st}_{k, \text{eid}, 2}$
$\text{st}_k \leftarrow \mathbf{Q}_{\text{st}}[k, \cdot, \cdot] \quad // \text{ all state for party } k$	$\text{return } \text{pm}_{k, \text{eid}, 2}$
$\text{return } (\text{sk}_k, \text{st}_k)$	$\mathcal{O}^{\text{Sign}_3}(k, \text{eid}, \{\text{pm}_{i, \text{eid}_i, 2}\}_{i \in \mathcal{S}})$
	$\text{return } \perp \text{ if } k \notin \text{hon}$
	$\text{return } \perp \text{ if } \mathbf{Q}_{\text{st}}[k, \text{eid}, 2] = \perp$
	$\text{return } \perp \text{ if } \mathbf{Q}_{\text{st}}[k, \text{eid}, 3] \neq \perp$
	$\text{parse } \text{st}_{k, \text{eid}, 2} \leftarrow \mathbf{Q}_{\text{st}}[k, \text{eid}, 2]$
	$\text{return } \perp \text{ if } \perp$
	$\leftarrow \text{Sign}_3(k, \text{st}_{k, \text{eid}, 2}, \{\text{pm}_{i, \text{eid}_i, 2}\}_{i \in \mathcal{S}})$
	$\text{pm}_{k, \text{eid}, 3}$
	$\leftarrow \text{Sign}_3(k, \text{st}_{k, \text{eid}, 2}, \{\text{pm}_{i, \text{eid}_i, 2}\}_{i \in \mathcal{S}})$
	$\mathbf{Q}_{\text{st}}[k, \text{eid}, 3] \leftarrow \text{pm}_{k, \text{eid}, 3}$
	$\text{return } \text{pm}_{k, \text{eid}, 3}$

Fig. 7. Static and adaptive unforgeability games for a threshold signature scheme with one pre-processing round followed by two online signing rounds. The public parameters par are implicitly given as input to all algorithms, and $\text{pm}_1, \text{pm}_2, \text{pm}_3$ represent protocol messages defined within the construction. The static game contains all but the dashed boxes, and the adaptive game adds the dashed boxes. frac specifies the fraction of t signers that the adversary may corrupt. For example, $\text{frac} = 1$ means the adversary can corrupt a full t signers, and $\text{frac} = 1/2$ means it can corrupt $t/2$ signers.

4.1 Definition of Static Security

Most threshold signature schemes in the literature are proven in the *static* security model, which captures a limited adversary that can only corrupt parties at the onset of the protocol. We provide a formal game-based definition in Fig. 7. The static security game contains all but the dashed boxes.

In the static unforgeability game, the environment accepts a security parameter κ and a corruption fraction **frac**. We consider a static adversary that can corrupt a full t (i.e., **frac** = 1) signers out of $t + 1$ signers in a session. The environment generates public parameters **par** and initializes the adversary with **par**. The adversary chooses a set of potential signers n , a threshold $t + 1$, and a set of corrupted parties **cor**. The environment checks that the number of corrupted parties does not exceed the maximum allowable amount $\text{frac} \cdot t$ and sets the honest parties $\text{hon} \leftarrow [n] \setminus \text{cor}$.

The environment then runs **KeyGen** to derive the threshold public key **pk** representing all n signers, the individual public key shares $\{\text{pk}_i\}_{i \in [n]}$, and the secret signing shares $\{\text{sk}_i\}_{i \in [n]}$. It returns **pk**, $\{\text{pk}_i\}_{i \in [n]}$, and the set of corrupt signing shares $\{\text{sk}_j\}_{j \in \text{cor}}$ to the adversary.

After key generation has concluded, the adversary can then query the various signing oracles $\mathcal{O}^{\text{Sign}_1, \text{Sign}_2, \text{Sign}_3}$ for honest signers to engage in subsequent signing rounds, across different sessions. The environment performs checks during the signing rounds to ensure each participant being called is in fact uncorrupted and in the correct signing round. Each check is carried out with respect to an execution identifier, denoted by **eid**, which is a random string sampled by the environment and bound to each new signing session for an honest party. Doing so enables the environment to distinguish concurrent signing sessions and update the correct session state for each subsequent signing round. Execution identifiers are akin to session identifiers (typically denoted as **sid**), however, we do *not* assume global consistency among signers for execution identifiers; each honest party samples and locally maintains their own **eid** for each signing session.

The adversary wins if it can produce a valid forgery $\sigma^* = (R^*, z^*)$ with respect to the threshold public key **pk** on a message m^* that has not been previously queried.

Note that our definition adds new state for each signing round. Specifically, the private state of signer k in session **eid** is stored in $\text{st}[k, \text{eid}, \text{rnd}]$, where **rnd** is the round. The adaptive security of **Sparkle+** does not rely on secure erasures: the state in each round is carried to all subsequent rounds.

We place minimal requirements on the communication network and do not assume pairwise authenticated channels between all parties (see Section 5.1). Thus, an adversary may send inconsistent protocol messages to honest parties, even on behalf of other honest parties.

The adversary may be *rushing*, i.e., it may wait to produce its outputs after having seen honest protocol messages in each round. Our model additionally captures a *concurrent* adversary, who may open simultaneous signing sessions at once or choose not to complete a signing session. We model concurrency explicitly

in our definition to ensure that our notion of security protects against practical attacks which can occur under a concurrent adversary [37, 17].

Definition 7 (Static Security). *Let the advantage of an adversary $\mathcal{A}$ playing the static security game, $\text{Game}_{\mathcal{A}, \text{TS}}^{\text{TS-EUF-CMA}}(\kappa, \text{frac})$, as defined in Figure 7, be as follows:*

$$\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{TS-EUF-CMA}}(\kappa, \text{frac}) = \Pr[\text{Game}_{\mathcal{A}, \text{TS}}^{\text{TS-EUF-CMA}}(\kappa, \text{frac}) = 1]$$

A threshold signature scheme TS is statically secure if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{TS-EUF-CMA}}(\kappa, \text{frac})$ is negligible.

4.2 Definition of Adaptive Security

We build upon the notion of adaptive security for threshold signature schemes by Libert, Joye, and Yung [53]. We provide a formal game-based definition, which is specified in Figure 7. The adaptive unforgeability game contains the same inputs and algorithms as the static game, but additionally includes a corruption oracle $\mathcal{O}^{\text{Corrupt}}$.

In the adaptive setting, the adversary is not restricted to choosing its set of corrupt participants cor only at the beginning of the game. Instead, the adversary can at any time choose to corrupt an honest party by querying $\mathcal{O}^{\text{Corrupt}}$, receiving in return the honest party's secret key and state across all signing sessions.

In addition to producing a valid forgery, the adversary must meet the winning condition that the set of corrupted participants at the end of the experiment is within the expected bound, i.e., less than $\text{frac} \cdot t$, with respect to the corruption ratio frac . An adaptive adversary may, for example, corrupt $t/2$ (i.e., $\text{frac} = 1/2$) or a full t (i.e., $\text{frac} = 1$) signers out of at least $t + 1$ signers in a session.

Definition 8 (Adaptive Security). *Let the advantage of an adversary $\mathcal{A}$ playing the adaptive security game $\text{Game}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-EUF-CMA}}(\kappa, \text{frac})$, as defined in Figure 7, be as follows:*

$$\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-EUF-CMA}}(\kappa, \text{frac}) = \Pr[\text{Game}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-EUF-CMA}}(\kappa, \text{frac}) = 1]$$

A threshold signature scheme TS is adaptively secure if for all PPT adversaries $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-EUF-CMA}}(\kappa, \text{frac})$ is negligible.

Definition 9 (Full Adaptive Security). *We say that a threshold signature scheme TS achieves full adaptive security if the value of the multiplier frac in $\text{Game}_{\mathcal{A}, \text{TS}}^{\text{adp-TS-EUF-CMA}}(\kappa, \text{frac})$ is $\text{frac} = 1$, i.e., the adversary is allowed to corrupt up to a full t participants.*

5 Threshold Schnorr Signature Scheme Sparkle+

Sparkle+ is a simple three-round threshold Schnorr signature scheme (Fig. 8). It consists of an offline phase, which may be executed before the message or signing

set are known, and an online phase consisting of two signing rounds. We assume an external mechanism to choose the set of signers $\mathcal{S} \subseteq [n]$, where $t+1 = |\mathcal{S}| \leq n$ and $\mathcal{S}$ is ordered to ensure consistency. We employ a centralized key generation algorithm; however, our scheme and proofs can be adapted to use a distributed key generation protocol (DKG). We refer to [6] for a discussion on how to achieve adaptively secure DKGs.

Intuitively, **Sparkle+** follows a commit-reveal paradigm for its three-round signing protocol. In the first round, each participant in the signing set $\mathcal{S}$ commits to their nonce $R_i = g^{r_i}$ by publishing $\text{cm}_i \leftarrow \text{H}_{\text{cm}}(i, R_i)$. In the second round, each participant reveals R_i in the clear, along with a DS signature on their protocol transcript. In the third round, each participant computes the aggregate nonce $R = \prod_{i \in \mathcal{S}} R_i$, the Schnorr challenge $c \leftarrow \text{H}_{\text{sig}}(\text{pk}, m, R)$ using R , and their signature share z_i . The signature shares are additively combined via the **Combine** algorithm to produce the Schnorr signature $\sigma = (R, z = \sum_{i \in \mathcal{S}} z_i)$.

This commit-reveal strategy ensures two security properties. First, requiring each participant to publish a commitment in the first round, before revealing their nonce R_i , prevents a rushing adversary from adaptively choosing their R_j as a function of other participants' R_i values, which is known to lead to ROS attacks [37, 17]. Second, as we will see later in the proofs of adaptive security, it allows the reduction to effectively handle corruptions at any point in the signing process, without requiring the erasure of secret state.

5.1 Communication Model

We assume that trusted key generation operations are performed over a reliable, authenticated, and private network communication channel. During key generation, parties may have authenticated channels with a coordinator, but are not assumed to have them with each other.

For communication between participants at the time of signing, we assume only that the network is reliable and do not assume consistency or authenticity of messages exchanged between participants. Thus, **Sparkle+** additionally makes use of a standard EUF-CMA-secure digital signature scheme **DS** to authenticate messages sent in the signing rounds. We place minimal requirements on the communication network for signing to give practitioners the flexibility to implement **Sparkle+** in myriad settings, without the risk of compromising security. This could result in some redundancy if, for example, there are pairwise authenticated channels between all parties.

5.2 Protocol Description

Parameter Generation. On input the security parameter 1^κ , the setup algorithm runs $(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\kappa)$ and selects two hash functions $\text{H}_{\text{cm}}, \text{H}_{\text{sig}} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$. It also runs the setup algorithm for a signature scheme $\text{par}_{\text{sig}} \leftarrow \text{DS.Setup}(1^\kappa)$ used for authentication in Signing Rounds 1 and 2. It outputs public parameters $\text{par} \leftarrow ((\mathbb{G}, p, g), \text{H}_{\text{cm}}, \text{H}_{\text{sig}}, \text{par}_{\text{sig}})$, which are provided as input to all other algorithms and protocols.

Setup (1^κ) $(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\kappa)$ $\quad \parallel$ select two hash functions $H_{\text{cm}}, H_{\text{sig}} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ $\text{par}_{\text{DS}} \leftarrow \text{DS.Setup}(1^\kappa)$ $\text{par} \leftarrow ((\mathbb{G}, p, g), H_{\text{cm}}, H_{\text{sig}}, \text{par}_{\text{DS}})$ return par	Sign₃ ($k, \text{st}_{k,2}, \{(R_i, \hat{\sigma}_i)\}_{i \in \mathcal{S}}$) parse $(R'_k, r_k, \text{cm}_k, \hat{\sigma}'_k, \mathcal{S}', m, \{\text{cm}_i\}_{i \in \mathcal{S}'}, \text{sk}_k)$ $\quad \leftarrow \text{st}_{k,2}$ return $\perp$ if $(R'_k, \hat{\sigma}'_k, \mathcal{S}') \neq (R_k, \hat{\sigma}_k, \mathcal{S})$ $\quad \parallel$ checks inputs against records in state for $i \in \mathcal{S}$ do $\quad$ return $\perp$ if $\text{cm}_i \neq H_{\text{cm}}(i, R_i)$ $\quad \text{msg}_i \leftarrow (i, \text{cm}_i, R_i, \mathcal{S}, m, \{\text{cm}_j\}_{j \in \mathcal{S}})$ $\quad$ if $\text{DS.Verify}(\hat{X}_i, \text{msg}_i, \hat{\sigma}_i) \neq 1$ $\quad \quad$ return $\perp$ $R \leftarrow \prod_{i \in \mathcal{S}} R_i$ $c \leftarrow H_{\text{sig}}(\text{pk}, m, R)$ $z_k \leftarrow r_k + c\lambda_k x_k$ $\quad \parallel \lambda_k$ is the Lagrange coefficient for k return z_k
KeyGen ($n, t + 1$) $x \leftarrow \mathbb{Z}_p; \text{pk} \leftarrow g^x$ $\{(i, x_i)\}_{i \in [n]} \leftarrow \text{IssueShares}(x, n, t + 1)$ $\quad \parallel$ Shamir secret sharing of x for $i \in [n]$ do $\quad X_i \leftarrow g^{x_i}; (\hat{X}_i, \hat{x}_i) \leftarrow \text{DS.KeyGen}()$ $\quad \text{pk}_i \leftarrow (X_i, \hat{X}_i)$ $\quad \text{sk}_i \leftarrow (x_i, \hat{x}_i, \text{pk}, \{\text{pk}_j\}_{j \in [n]})$ return $(\text{pk}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [n]})$	Combine ($\mathcal{S}, m, \{\text{cm}_i, (R_i, \hat{\sigma}_i), z_i\}_{i \in \mathcal{S}}$) $R \leftarrow \prod_{i \in \mathcal{S}} R_i; z \leftarrow \sum_{i \in \mathcal{S}} z_i$ $\sigma \leftarrow (R, z)$ return σ
Sign₁ (k, sk_k) $r_k \leftarrow \mathbb{Z}_p; R_k \leftarrow g^{r_k}$ $\text{cm}_k \leftarrow H_{\text{cm}}(k, R_k)$ $\text{st}_{k,1} \leftarrow (\text{cm}_k, R_k, r_k, \text{sk}_k)$ return $(\text{cm}_k, \text{st}_{k,1})$	Verify (pk, m, σ) parse $(R, z) \leftarrow \sigma$ $c \leftarrow H_{\text{sig}}(\text{pk}, m, R)$ return 1 if $R \cdot \text{pk}^c = g^z$ return 0
Sign₂ ($k, \text{st}_{k,1}, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}$) return $\perp$ if $\mathcal{S} \not\subseteq [n] \vee \mathcal{S} \leq t + 1$ $\vee k \notin \mathcal{S}$ parse $(\text{cm}'_k, R_k, r_k, \text{sk}_k) \leftarrow \text{st}_{k,1}$ $\quad \parallel$ checks inputs against records in state return $\perp$ if $\text{cm}'_k \neq \text{cm}_k$ $\text{msg}_k \leftarrow (k, \text{cm}_k, R_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$ $\hat{\sigma}_k \leftarrow \text{DS.Sign}(\hat{x}_k, \text{msg}_k)$ $\text{st}_{k,2}$ $\quad \leftarrow (R_k, r_k, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \text{sk}_k)$ return $((R_k, \hat{\sigma}_k), \text{st}_{k,2})$	

Fig. 8. Sparkle+ threshold signature scheme with one round of pre-processing. Consistency checks on the state, e.g., ensuring that $\text{st}_{k,1}$ is only used once, are enforced in the definition of unforgeability (Fig. 7) and implementers must additionally include these checks. The public parameters par are implicitly given as input to all algorithms. We assume an external mechanism to choose the set of signers $\mathcal{S} \subseteq [n]$, where $|\mathcal{S}| = t + 1$ and $\mathcal{S}$ is ordered to ensure consistency. DS is a EUF-CMA-secure digital signature scheme (e.g., a Schnorr signature). Verification is identical to the Schnorr signature scheme as in Definition 5.

Key Generation. On input the number of signers n and the threshold $t + 1$, the key generation algorithm first generates the secret key $x \leftarrow^s \mathbb{Z}_p$ and threshold public key $\text{pk} \leftarrow g^x$. It then performs a Shamir secret sharing of x : $\{(i, x_i)\}_{i \in [n]} \leftarrow^s \text{IssueShares}(x, n, t + 1)$ (Def. 1). It computes the corresponding public key for each participant as $X_i \leftarrow g^{x_i}$. It then runs the key generation algorithm $(\hat{X}_i, \hat{x}_i) \leftarrow \text{DS.KeyGen}()$. It sets $\text{pk}_i \leftarrow (X_i, \hat{X}_i)$ and $\text{sk}_i \leftarrow (x_i, \hat{x}_i, \text{pk}, \{\text{pk}_i\}_{i \in [n]})$. Finally, it outputs $(\text{pk}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [n]})$.

Signing Round 1 (Sign₁). In the first round, a potential signer samples $r_i \leftarrow^s \mathbb{Z}_p$, computes $R_i \leftarrow g^{r_i}$ and $\text{cm}_i \leftarrow \text{H}_{\text{cm}}(i, R_i)$, and outputs commitment cm_i .

Signing Round 2 (Sign₂). In the second round, each signer reveals their committed nonce R_i . On input a message m , a signing set $\mathcal{S}$, and a set of commitments $\{\text{cm}_j\}_{j \in \mathcal{S}}$, each participant $i \in \mathcal{S}$ checks that the commitment given as input on behalf of them, i.e., $\text{cm}_i \in \{\text{cm}_j\}_{j \in \mathcal{S}}$, is in fact what is recorded in their state from Signing Round 1. They then form the message $\text{msg}_i \leftarrow (i, \text{cm}_i, R_i, \mathcal{S}, m, \{\text{cm}_j\}_{j \in \mathcal{S}})$ and run the signing algorithm $\hat{\sigma}_i \leftarrow \text{DS.Sign}(\hat{x}_i, \text{msg}_i)$, used to authenticate the messages sent in Signing Rounds 1 and 2. Each signer then outputs their committed nonce R_i and signature $\hat{\sigma}_i$.

Signing Round 3 (Sign₃). In the third and final round, each signer outputs their partial signature z_i . On input a set of nonce-signature pairs $\{(R_j, \hat{\sigma}_j)\}_{j \in \mathcal{S}}$, each participant $i \in \mathcal{S}$ first checks that the signing set $\mathcal{S}$ and nonce-signature pair given as input on behalf of them, i.e., $(R_i, \hat{\sigma}_i) \in \{(R_j, \hat{\sigma}_j)\}_{j \in \mathcal{S}}$, is in fact what is recorded in their state from Signing Round 2. Each participant then checks that the commitments received in the first round are valid, i.e., if for some $j \in \mathcal{S}$, $\text{cm}_j \neq \text{H}_{\text{cm}}(j, R_j)$, return $\perp$. They also form messages $\text{msg}_j \leftarrow (j, \text{cm}_j, R_j, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$ for all $j \in \mathcal{S}$ and check that all signatures $\hat{\sigma}_j$ verify: $\text{DS.Verify}(\hat{X}_j, \text{msg}_j, \hat{\sigma}_j) = 1$ and return $\perp$ if not. The signatures ensures that the honest signing parties all agree on the shared view of the message, signing set, and protocol messages sent in Signing Rounds 1 and 2. If all checks pass, each participant computes the aggregate nonce $R = \prod_{j \in \mathcal{S}} R_j$, $c \leftarrow \text{H}_{\text{sig}}(\text{pk}, m, R)$, and partial signature $z_i \leftarrow r_i + c\lambda_i x_i$, where λ_i is the Lagrange coefficient for participant i with respect to signing set $\mathcal{S}$. Finally, each signer outputs z_i .

Combining Signatures. On input a signing set $\mathcal{S}$, message m , commitments $\{\text{cm}_j\}_{j \in \mathcal{S}}$, nonces $\{R_j\}_{j \in \mathcal{S}}$, and partial signatures $\{z_j\}_{j \in \mathcal{S}}$, the combiner computes $R \leftarrow \prod_{j \in \mathcal{S}} R_j$ and $z \leftarrow \sum_{j \in \mathcal{S}} z_j$, and outputs $\sigma \leftarrow (R, z)$.

Verification. On input the threshold public key pk , a message m , and a purported signature $\sigma = (R, z)$, the verifier computes $c \leftarrow \text{H}_{\text{sig}}(\text{pk}, m, R)$ and accepts if $R \cdot \text{pk}^c = g^z$.

Correctness of **Sparkle+** is straightforward to verify. Note that verification of the signature σ is identical to verification of a standard (single-party) Schnorr signature (Def. 5) with respect to the threshold public key $\mathbf{pk}$ and aggregate nonce R .

6 Our Results

6.1 Static Security Under Standard Assumptions

Our first result is that **Sparkle+** is statically secure under the EUF-CMA security of DS and under the discrete logarithm assumption (DL) in the random oracle model (ROM). DS itself may, for example, be a Schnorr signature, which is secure under DL in the ROM [60]. We allow a static adversary to control up to t parties, but they must be declared at the beginning of the protocol (Fig. 7).

Theorem 1. *Let GrGen be a group generator and DS be a digital signature scheme. For any PPT adversary $\mathcal{A}$ for the game $\text{Game}_{\mathcal{A}, \text{Sparkle}^+}^{\text{TS-EUF-CMA}}(\kappa, \text{frac} = 1)$ (Fig. 7), there exists a PPT adversary $\mathcal{B}_1$ for the game $\text{Game}_{\mathcal{B}_1, \text{DS}}^{\text{EUF-CMA}}(\kappa)$ (Fig. 4) making at most q_S queries to its signing oracle, and a PPT adversary $\mathcal{B}_2$ for the game $\text{Game}_{\mathcal{B}_2}^{\text{dl}}(\kappa)$ (Fig. 5) such that*

$$\text{Adv}_{\mathcal{A}, \text{Sparkle}^+}^{\text{TS-EUF-CMA}}(\kappa) \leq \sqrt{q \cdot \text{Adv}_{\mathcal{B}_2}^{\text{dl}}(\kappa) + \frac{q^2}{2p} + \frac{2q}{p} + 2n \cdot \text{Adv}_{\mathcal{B}_1, \text{DS}}^{\text{EUF-CMA}}(\kappa)} \quad (3)$$

where n is the number of signers, t is the maximum number of corrupt signers, p is the group size, and $q = q_H + q_S + 1$, where q_H is the number of queries $\mathcal{A}$ can make to the random oracles, and q_S is the number of queries $\mathcal{A}$ can make to the signing oracles.

We formally prove Theorem 1 in Appendix A.

Proof Outline. Let $\mathcal{A}$ be a PPT adversary against the static unforgeability of **Sparkle+** (Fig. 7). We construct a PPT reduction $\mathcal{B}_1$ against the EUF-CMA security of the signature scheme DS and a PPT reduction $\mathcal{B}_2$ against the DL assumption (Fig. 5). We show that if $\mathcal{A}$ produces a forgery with non-negligible probability, then either $\mathcal{B}_1$ breaks the EUF-CMA security of DS, or $\mathcal{B}_2$ breaks the DL assumption with non-negligible probability.

The reduction $\mathcal{B}_1$ takes as input a challenge public key $\hat{X}^\dagger$ for DS and simply passes it to $\mathcal{A}$ as one of the n honest parties' public keys during key generation for DS. $\mathcal{B}_1$ runs trusted key generation for **Sparkle+** honestly. If $\mathcal{A}$ forges any signature, then the probability that $\mathcal{A}$ forges a DS signature for this select party in Round 2 is greater than $1/n$.

The reduction $\mathcal{B}_2$ takes as input a DL challenge $\dot{X}$ and aims to output $\dot{x}$ such that $\dot{X} = g^{\dot{x}}$. $\mathcal{B}_2$ sets the threshold public key $\mathbf{pk} \leftarrow \dot{X}$ and performs a standard simulation of Shamir secret sharing, resulting in public key shares X_i for all $i \in [n]$.

$\mathcal{B}_2$ runs key generation for DS honestly, resulting in public key shares $\hat{X}_i = g^{\hat{x}_i}$ for all $i \in [n]$. $\mathcal{B}_2$ sets $\text{pk}_i \leftarrow (X_i, \hat{X}_i)$ and $\text{sk}_i \leftarrow (x_i, \hat{x}_i)$ for all $i \in [n]$ and returns pk , $\{\text{pk}_i\}_{i \in [n]}$, and $\{\text{sk}_j\}_{j \in \text{cor}}$ to $\mathcal{A}$.

$\mathcal{B}_2$ simulates Sparkle+ signing without knowing the secret keys $\{x_k\}_{k \in \text{hon}}$ of the honest parties as follows. (Our simulation technique is actually quite technical, and we outline a simplified approach here.) $\mathcal{B}_2$ can simulate R_k for all $k \in \text{hon}$ as $R_k \leftarrow g^{z_k} X_k^{-c\lambda_k}$ for random $z_k \leftarrow \mathbb{Z}_p$ so that the partial signature z_k output in Round 3 verifies. However, c must equal $H_{\text{sig}}(\text{pk}, m, R)$ for aggregate R . Luckily, $\mathcal{B}_2$ can compute $R = \prod_{i \in \mathcal{S}} R_i$, where $\mathcal{S}$ is the signing set, by extracting R_j for all $j \in \text{cor}$ from $\mathcal{A}$'s $H_{\text{cm}}(j, R_j)$ queries in Round 1. So, $\mathcal{B}_2$ samples a random value for $c \leftarrow \mathbb{Z}_p$, computes R_k for all $k \in \text{hon}$ as above, and programs $c \leftarrow H_{\text{sig}}(\text{pk}, m, R)$ – all before $\mathcal{A}$ sees the honest R_k 's output at the end of Round 2. This leaves just one problem: $\mathcal{B}_2$ needs to output $\text{cm}_k = H_{\text{cm}}(k, R_k)$ in Round 1 *before* being able to carry out the above steps. But $\mathcal{B}_2$ can simply output a random value $\text{cm}_k \leftarrow \mathbb{Z}_p$ in Round 1 and program $\text{cm}_k \leftarrow H_{\text{cm}}(k, R_k)$ in Round 2.

$\mathcal{B}_2$ then *rewinds* $\mathcal{A}$, using the general forking strategy of Bellare and Neven [12]. With non-negligible probability, $\mathcal{A}$ will output two forgeries across its two iterations $(m^*, (R^*, z^*)), (m', (R', z'))$ that satisfy $(R^*, m^*) = (R', m')$, but where $c^* \neq c'$. Thus, $\mathcal{B}_2$ can compute $\hat{x} = \frac{z^* - z'}{c^* - c'}$ and win the DL game.

6.2 Adaptive Security up to $t/2$ Corruptions

Our second result is that Sparkle+ is adaptively secure against up to $t/2$ corruptions under the EUF-CMA security of DS and under the algebraic one-more discrete logarithm assumption (AOMDL) (Fig. 5) in the random oracle model (ROM). The reason the allowed corruption is $t/2$ is that, in order to extract an AOMDL solution, the reduction needs to rewind the adversary once, and there is no guarantee that the adversary will corrupt the same set of parties after the fork as it did during its first execution. When the adversary can corrupt only $t/2$ parties, this causes no issues, as the total number of corruptions over both iterations does not exceed t . If the adversary could corrupt more parties, then the reduction would query its discrete logarithm oracle more than t times and would lose the $t + 1$ -aomdl game.

Theorem 2. *Let GrGen be a group generator and DS be a digital signature scheme. For any PPT adversary $\mathcal{A}$ for the adaptive unforgeability game $\text{Game}_{\mathcal{A}, \text{Sparkle}^+}^{\text{adp-TS-EUF-CMA}}(\kappa, \text{frac} = 1/2)$ (Fig. 7), there exists a PPT adversary $\mathcal{B}'_1$ for the game $\text{Game}_{\mathcal{B}'_1, \text{DS}}^{\text{EUF-CMA}}(\kappa)$ (Fig. 4) making at most q_S queries to its signing oracle, and a PPT adversary $\mathcal{B}'_2$ for the game $\text{Game}_{\mathcal{B}'_2}^{t+1\text{-aomdl}}(\kappa)$ (Fig. 5) such that*

$$\text{Adv}_{\mathcal{A}, \text{Sparkle}^+}^{\text{adp-TS-EUF-CMA}}(\kappa, 1/2) \leq \sqrt{q \text{Adv}_{\mathcal{B}'_2}^{t+1\text{-aomdl}}(\kappa) + \frac{q^2}{2p} + \frac{5q}{p} + \frac{n^2}{(n-t/2)} \text{Adv}_{\mathcal{B}'_1, \text{DS}}^{\text{EUF-CMA}}(\kappa)} \quad (4)$$

where n is the number of signers, $t/2$ is the maximum number of corrupted signers, p is the group size, and $q = q_H + q_S + 1$, where q_H is the number of

queries $\mathcal{A}$ can make to the random oracles, and q_S is the number of queries $\mathcal{A}$ can make to the signing oracles.

We formally prove Theorem 2 in Appendix B.

Proof Outline. Let $\mathcal{A}$ be a PPT adversary against the $t/2$ adaptive unforgeability of Sparkle+ (Fig. 7). We construct a PPT reduction $\mathcal{B}'_1$ against the EUF-CMA security of the signature scheme DS and a PPT reduction $\mathcal{B}'_2$ against the $t+1$ -aomdl assumption (Fig. 5). We show that if $\mathcal{A}$ produces a forgery with non-negligible probability, then either $\mathcal{B}'_1$ breaks the EUF-CMA security of DS, or $\mathcal{B}'_2$ breaks the $t+1$ -aomdl assumption with non-negligible probability. The reduction $\mathcal{B}'_1$ is the same as in the static security proof.

The reduction $\mathcal{B}'_2$ takes as input a $t+1$ -aomdl challenge $(Y_0, Y_1, \dots, Y_t)$ and aims to output y_i such that $Y_i = g^{y_i}$ for all $i \in [0..t]$ without querying its discrete logarithm solution oracle more than t times. For all $i \in [n]$, $\mathcal{B}'_2$ sets each Sparkle+ public key share as $X_i \leftarrow Y_0 Y_1^i \dots Y_t^{i^t}$. The threshold public key is $\text{pk} \leftarrow Y_0$.

$\mathcal{B}'_2$ runs key generation for DS honestly, resulting in public key shares $\hat{X}_i = g^{\hat{x}_i}$ for all $i \in [n]$. $\mathcal{B}'_2$ sets $\text{pk}_i \leftarrow (X_i, \hat{X}_i)$ and $\text{sk}_i \leftarrow (x_i, \hat{x}_i)$ for all $i \in [n]$. $\mathcal{B}'_2$ queries its DL oracle on X_j (with representation $(1, j, \dots, j^t)$) to get x_j for the initial corrupted set cor . $\mathcal{B}'_2$ then returns $(\text{pk}, \{\text{pk}_i\}_{i \in [n]}, \{\text{sk}_j\}_{j \in \text{cor}})$ to $\mathcal{A}$.

$\mathcal{B}'_2$'s simulation of signing is similar to the proof of static security. (This is by design, we use the same simulation techniques, even if they are not strictly necessary in the static setting, in order to modularize the proofs. Even so, the adaptive proofs are considerably more technical.) In particular, $\mathcal{B}'_2$ again simulates R_k for honest $k \in \text{hon}$ as $R_k \leftarrow g^{z_k} X_k^{-c\lambda_k}$ in Round 2. If $\mathcal{A}$ corrupts an honest party k after Round 2 has begun, then $\mathcal{B}'_2$ queries its DL oracle on X_k (with rep. $(1, k, \dots, k^t)$) to get x_k , computes $r_k \leftarrow z_k - c\lambda_k x_k$, and returns $x_k, \hat{x}_k$, and the honest party's state, including nonces, commitments, etc. across all signing sessions to $\mathcal{A}$. However, $\mathcal{A}$ may choose to corrupt some honest k before Round 2, or even before it outputs its own cm_j 's in Round 1. In this case, $\mathcal{B}'_2$ samples a random $r_k \leftarrow \mathbb{Z}_p$, sets $R_k \leftarrow g^{r_k}$, and programs $\text{cm}_k \leftarrow \text{H}_{\text{cm}}(k, R_k)$ for the random cm_k it output in Round 1. It then queries its DL oracle on X_k (with rep. $(1, k, \dots, k^t)$) to get x_k and returns it, $\hat{x}_k$, and state across all signing sessions to $\mathcal{A}$. As in the proof of static security, $\mathcal{B}'_2$ rewinds $\mathcal{A}$ in order to extract $x = y_0$ from $\mathcal{A}$'s two forgeries. Assume w.l.o.g. that $\mathcal{A}$ corrupts t parties over the two iterations. ($\mathcal{A}$ can corrupt up to $t/2$ parties in each iteration, and $\mathcal{B}'_2$ can corrupt the remaining itself.) For simplicity, say the corrupt indices are $\text{cor} = \{1, \dots, t\}$. Then $\mathcal{B}'_2$ has made t DL queries on $g^{x_k} = Y_0 Y_1^k \dots Y_t^{k^t}$. This forms a system of linear equations:

$$\begin{pmatrix} x \\ x_1 \\ \vdots \\ x_t \end{pmatrix} = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 1 & 1 & \dots & 1^t \\ \vdots & \vdots & \ddots & \vdots \\ 1 & t & \dots & t^t \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_t \end{pmatrix}$$

This is a Vandermonde matrix and is therefore invertible. $\mathcal{B}'_2$ knows $(x, x_1, \dots, x_t)$, and so can solve for $(y_0, y_1, \dots, y_t)$ to win the $t+1$ -aomdl game.

6.3 Adaptive Security up to t Corruptions

We now present our main result: **Sparkle+** is secure against a full t adaptive corruptions. In particular, if exactly $t + 1$ parties engage in signing, all but one of them could be malicious and the unforgeability of **Sparkle+** would still hold. We prove this result under the EUF-CMA security of DS and under the AOMDL assumption (Fig. 5) in the AGM with random oracles.

Theorem 3. *Let GrGen be a group generator and DS be a digital signature scheme. For any algebraic PPT adversary $\mathcal{A}$ for the adaptive unforgeability game $\text{Game}_{\mathcal{A}, \text{Sparkle}^+}^{\text{adp-TS-EUF-CMA}}(\kappa, \text{frac} = 1)$ (Fig. 7), there exists a PPT adversary $\mathcal{B}'_1$ for the game $\text{Game}_{\mathcal{B}'_1, \text{DS}}^{\text{EUF-CMA}}(\kappa)$ (Fig. 4) making at most q_S queries to its signing oracle, and a PPT adversary $\mathcal{B}''_2$ for the game $\text{Game}_{\mathcal{B}''_2}^{t+1\text{-aomdl}}(\kappa)$ (Fig. 5) such that*

$$\text{Adv}_{\mathcal{A}, \text{Sparkle}^+}^{\text{adp-TS-EUF-CMA}}(\kappa, 1) \leq \text{Adv}_{\mathcal{B}''_2}^{t+1\text{-aomdl}}(\kappa) + \frac{q^2}{2p} + \frac{5q}{p} + \frac{n^2}{(n-t)} \text{Adv}_{\mathcal{B}'_1, \text{DS}}^{\text{EUF-CMA}}(\kappa) \quad (5)$$

where n is the number of signers, t is the maximum number of corrupted signers, p is the group size, and $q = q_H + q_S + 1$, where q_H is the number of queries $\mathcal{A}$ can make to the random oracles, and q_S is the number of queries $\mathcal{A}$ can make to the signing oracles.

We formally prove Theorem 3 in Appendix C.

Proof Outline. Let $\mathcal{A}$ be a PPT adversary against the full adaptive unforgeability of **Sparkle+** (Fig. 7). We construct a PPT reduction $\mathcal{B}'_1$ against the EUF-CMA security of the signature scheme DS and a PPT reduction $\mathcal{B}''_2$ against the $t + 1$ -aomdl assumption (Fig. 5). We show that if $\mathcal{A}$ produces a forgery with non-negligible probability, then either $\mathcal{B}'_1$ breaks the EUF-CMA security of DS, or $\mathcal{B}''_2$ breaks the $t + 1$ -aomdl assumption with non-negligible probability. The reduction $\mathcal{B}'_1$ is the same as in the static security proof.

The reduction $\mathcal{B}''_2$ takes as input a $t + 1$ -aomdl challenge $(Y_0, Y_1, \dots, Y_t)$ and aims to output y_i such that $Y_i = g^{y_i}$ for all $i \in [0..t]$ without querying its DL oracle more than t times. $\mathcal{B}''_2$ simulates key generation, signing, and corruption as in the $t/2$ -adaptive proof, but does not rewind $\mathcal{A}$, so $\mathcal{A}$ may corrupt a full t parties.

$\mathcal{A}$'s forgery $(m^*, (R^*, z^*))$ verifies as $R^* = g^{z^*} \text{pk}^{-c^*}$, where $\mathcal{A}$ provided a representation of R^* when querying $c^* = \text{H}_{\text{sig}}(\text{pk}, m^*, R^*)$:

$$R^* = g^{\gamma^*} \text{pk}^{\xi^*} X_1^{\xi_1^*} \dots X_n^{\xi_n^*} \prod_{i=1}^{q_S} R_{i,1}^{\rho_{i,1}^*} \dots R_{i,n}^{\rho_{i,n}^*}$$

where $\{R_{i,k}\}_{i \in [q_S]}$ are the honest nonces returned by the $\mathcal{O}^{\text{Sign}_2}$ oracle over q_S signing queries and $\rho_{i,k}^* = 0$ for k not in the signing set for query i . Each $R_{i,k}$ verifies as $R_{i,k} = g^{z_{i,k}} X_k^{-c_i \lambda_{i,k}}$, where $c_i = \text{H}_{\text{sig}}(\text{pk}, m_i, R_i)$. Equating the two expressions for R^* and rearranging, we have:

$$\begin{aligned}
g^{z^*} g^{-\gamma^*} \prod_{i=1}^{qS} g^{-z_{i,1} \rho_{i,1}^*} \dots g^{-z_{i,n} \rho_{i,n}^*} \\
= \text{pk}^{c^*} \text{pk}^{\xi^*} X_1^{\xi_1^*} \dots X_n^{\xi_n^*} \prod_{i=1}^{qS} (X_1^{-c_i \lambda_{i,1}})^{\rho_{i,1}^*} \dots (X_n^{-c_i \lambda_{i,n}})^{\rho_{i,n}^*}
\end{aligned}$$

$\mathcal{B}_2''$ queries its DL oracle on X_j (with rep. $(1, j, \dots, j^t)$) to obtain $\{x_j\}_{j \in \text{cor}}$ for the t corrupt parties. For all $k \in [n]$, $X_k = \text{pk} Y_1^k Y_2^{k^2} \dots Y_t^{k^t}$, and for all $i \in [t]$, $Y_i = \text{pk}^{L_{0,i}} \prod_{j \in \text{cor}} g^{x_j L'_{j,i}}$, where $L'_{j,i}$ is the i^{th} coefficient of the Lagrange polynomial $L'_j(Z)$ for the set $0 \cup \text{cor}$. Plugging these in and rearranging, we have:

$$g^{\eta^*} = \text{pk}^{c^* + \xi^*} \prod_{k=1}^n X^{\mu_k^*} g^{\nu_k^*}$$

where $\eta^* = z^* - \gamma^* - \sum_{i=1}^{qS} (z_{i,1} \rho_{i,1}^* + \dots + z_{i,n} \rho_{i,n}^*)$, $\mu_k^* = 1 + \sum_{i=1}^t L'_{0,i} k^i$, and $\nu_k^* = \sum_{i=1}^t \left(\sum_{j \in \text{cor}} x_j L'_{j,i} \right) k^i$. Then:

$$x = \frac{\eta^* - \sum_{k=1}^n \nu_k^*}{c^* + \xi^* + \sum_{k=1}^n \mu_k^*}$$

A fixed R^* and thus $\eta^*, \{\nu_i^*\}_{i \in [n]}, \xi^*, \{\mu_i^*\}_{i \in [n]}$ as it queried $\text{H}_{\text{sig}}(\text{pk}, m^*, R^*)$ to receive random c^* . Thus, the denominator is nonzero with overwhelming probability and $\mathcal{B}_2''$ can solve for x . $\mathcal{B}_2''$ can then compute $(y_0, y_1, \dots, y_t)$ as in the $t/2$ -adaptive proof to win the $t+1$ -aomdl game.

7 Conclusion

In this work, we present **Sparkle+**, an efficient and practical threshold Schnorr signature scheme with one round of pre-processing and two online signing rounds. We show that **Sparkle+** achieves several levels of security based on different corruption models and assumptions. For static security, we show that **Sparkle+** can be proven secure under the DL assumption in the ROM. We then show that **Sparkle+** can be proven adaptively secure under $t/2$ corruptions, assuming AOMDL in the ROM. Finally, we show that **Sparkle+** can be proven fully adaptively secure (with up to t corruptions), assuming AOMDL in the ROM and AGM.

Our work leaves open the intriguing question of under what conditions threshold signatures can be proven adaptively secure, and the extent to which threshold Schnorr signatures can achieve adaptive security under weaker security assumptions.

References

- [1] M. Abdalla, M. Barbosa, J. Katz, J. Loss, and J. Xu. “Algebraic Adversaries in the Universal Composability Framework”. In: *Advances in Cryptology - ASIACRYPT*

- 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part III. Ed. by M. Tibouchi and H. Wang. Vol. 13092. Lecture Notes in Computer Science. Springer, 2021, pp. 311–341.
- [2] M. Abe and S. Fehr. “Adaptively Secure Feldman VSS and Applications to Universally-Composable Threshold Cryptography”. In: *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*. Ed. by M. K. Franklin. Vol. 3152. Lecture Notes in Computer Science. Springer, 2004, pp. 317–334. DOI: 10.1007/978-3-540-28628-8_20. URL: https://doi.org/10.1007/978-3-540-28628-8_20.
 - [3] J. F. Almansa, I. Damgård, and J. B. Nielsen. “Simplified Threshold RSA with Adaptive and Proactive Security”. In: *EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006*. Ed. by S. Vaudenay. Vol. 4004. LNCS. Springer, 2006, pp. 593–611.
 - [4] H. K. Alper and J. Burdges. “Two-Round Trip Schnorr Multi-signatures via Delinearized Witnesses”. In: *CRYPTO 2021, Virtual Event, August 16-20, 2021*. Ed. by T. Malkin and C. Peikert. Vol. 12825. LNCS. Springer, 2021, pp. 157–188.
 - [5] R. Bacho, S. Das, J. Loss, and L. Ren. “Glacius: Threshold Schnorr Signatures from DDH with Full Adaptive Security”. In: *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part II*. Ed. by S. Fehr and P. Fouque. Vol. 15602. Lecture Notes in Computer Science. Springer, 2025, pp. 304–334. DOI: 10.1007/978-3-031-91124-8_11. URL: https://doi.org/10.1007/978-3-031-91124-8_11.
 - [6] R. Bacho and J. Loss. “On the Adaptive Security of the Threshold BLS Signature Scheme”. In: *IACR Cryptol. ePrint Arch.* (2022), p. 534. URL: <https://eprint.iacr.org/2022/534>.
 - [7] R. Bacho, J. Loss, G. Stern, and B. Wagner. “HARTS: High-Threshold, Adaptively Secure, and Robust Threshold Schnorr Signatures”. In: *IACR Cryptol. ePrint Arch.* (2024), p. 280. URL: <https://eprint.iacr.org/2024/280>.
 - [8] R. Bacho, J. Loss, S. Tessaro, B. Wagner, and C. Zhu. “Twinkle: Threshold Signatures from DDH with Full Adaptive Security”. In: *IACR Cryptol. ePrint Arch.* (2023), p. 1482. URL: <https://eprint.iacr.org/2023/1482>.
 - [9] B. Bauer, G. Fuchsbaauer, and A. Plouviez. “The One-More Discrete Logarithm Assumption in the Generic Group Model”. In: *ASIACRYPT 2021, Singapore, December 6-10, 2021*. Ed. by M. Tibouchi and H. Wang. Vol. 13093. LNCS. Springer, 2021, pp. 587–617.
 - [10] M. Bellare, E. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu. *Better than advertised security for non-interactive threshold signatures*. CRYPTO 2022. To appear. 2022.
 - [11] M. Bellare, C. Namprempre, D. Pointcheval, and M. Semanko. “The One-More-RSA-Inversion Problems and the Security of Chaum’s Blind Signature Scheme”. In: *J. Cryptol.* 16.3 (2003), pp. 185–215.
 - [12] M. Bellare and G. Neven. “Multi-signatures in the plain public-Key model and a general forking lemma”. In: *CCS 2006, Alexandria, VA, USA, October 30 - November 3, 2006*. Ed. by A. Juels, R. N. Wright, and S. D. C. di Vimercati. ACM, 2006, pp. 390–399.

- [13] M. Bellare and A. Palacio. “GQ and Schnorr Identification Schemes: Proofs of Security against Impersonation under Active and Concurrent Attacks”. In: *CRYPTO 2002, USA, August 18-22, 2002, Proceedings*. Ed. by M. Yung. Vol. 2442. LNCS. Springer, 2002, pp. 162–177. DOI: 10.1007/3-540-45708-9_11. URL: https://doi.org/10.1007/3-540-45708-9_11.
- [14] M. Bellare and P. Rogaway. “Random Oracles are Practical: A Paradigm for Designing Efficient Protocols”. In: *CCS '93, Proceedings of the 1st ACM Conference on Computer and Communications Security, Fairfax, Virginia, USA, November 3-5, 1993*. Ed. by D. E. Denning, R. Pyle, R. Ganesan, R. S. Sandhu, and V. Ashby. ACM, 1993, pp. 62–73. DOI: 10.1145/168588.168596. URL: <https://doi.org/10.1145/168588.168596>.
- [15] M. Bellare and S. Shoup. “Two-Tier Signatures, Strongly Unforgeable Signatures, and Fiat-Shamir Without Random Oracles”. In: *PKC 2007, Beijing, China, April 16-20, 2007, Proceedings*. Ed. by T. Okamoto and X. Wang. Vol. 4450. LNCS. Springer, 2007, pp. 201–216. DOI: 10.1007/978-3-540-71677-8_14. URL: https://doi.org/10.1007/978-3-540-71677-8_14.
- [16] M. Bellare, S. Tessaro, and C. Zhu. “Stronger Security for Non-Interactive Threshold Signatures”. In: *IACR Cryptol. ePrint Arch.* (2022), p. 833. URL: <https://eprint.iacr.org/2022/833>.
- [17] F. Benhamouda, T. Lepoint, J. Loss, M. Orrù, and M. Raykova. “On the (in)Security of ROS”. In: *J. Cryptol.* 35.4 (2022), p. 25. DOI: 10.1007/S00145-022-09436-0. URL: <https://doi.org/10.1007/s00145-022-09436-0>.
- [18] A. Boldyreva. “Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme”. In: *PKC 2003, Miami, FL, USA, January 6-8, 2003*. Ed. by Y. Desmedt. Vol. 2567. LNCS. Springer, 2003, pp. 31–46.
- [19] D. Boneh, M. Drijvers, and G. Neven. “Compact Multi-signatures for Smaller Blockchains”. In: *ASIACRYPT 2018, Brisbane, QLD, Australia, December 2-6, 2018*. Ed. by T. Peyrin and S. D. Galbraith. Vol. 11273. LNCS. Springer, 2018, pp. 435–464.
- [20] D. Boneh and V. Shoup. *A Graduate Course in Applied Cryptography*. 2020. URL: <http://toc.cryptobook.us/book.pdf>.
- [21] L. Brandão and M. Davidson. *Notes on Threshold EdDSA/Schnorr Signatures*. <https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8214B.ipd.pdf>. 2022.
- [22] L. Brandão and R. Peralta. *NIST First Call for Multi-Party Threshold Schemes*. <https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8214C.ipd.pdf>. 2023.
- [23] R. Canetti. “Universally Composable Security: A New Paradigm for Cryptographic Protocols”. In: *FOCS 2001, 14-17 October 2001, Las Vegas, Nevada, USA*. IEEE Computer Society, 2001, pp. 136–145.
- [24] R. Canetti, U. Feige, O. Goldreich, and M. Naor. “Adaptively Secure Multi-Party Computation”. In: *STOC '96, Philadelphia, Pennsylvania, USA, May 22-24, 1996*. Ed. by G. L. Miller. ACM, 1996, pp. 639–648.
- [25] R. Canetti, R. Gennaro, S. Goldfeder, N. Makriyannis, and U. Peled. “UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts”. In: *IACR Cryptol. ePrint Arch.* (2021), p. 60. URL: <https://eprint.iacr.org/2021/060>.
- [26] R. Canetti, R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. “Adaptive Security for Threshold Cryptosystems”. In: *CRYPTO '99, Santa Barbara, California, USA,*

- August 15-19, 1999. Ed. by M. J. Wiener. Vol. 1666. LNCS. Springer, 1999, pp. 98–115.
- [27] D. Chaum and T. P. Pedersen. “Wallet Databases with Observers”. In: *Advances in Cryptology - CRYPTO '92, 12th Annual International Cryptology Conference, Santa Barbara, California, USA, August 16-20, 1992, Proceedings*. Ed. by E. F. Brickell. Vol. 740. Lecture Notes in Computer Science. Springer, 1992, pp. 89–105. DOI: 10.1007/3-540-48071-4_7. URL: https://doi.org/10.1007/3-540-48071-4_7.
 - [28] Y. Chen. “Round-Efficient Adaptively Secure Threshold Signatures with Rewinding”. In: *IACR Cryptol. ePrint Arch.* (2025), p. 638. URL: <https://eprint.iacr.org/2025/638>.
 - [29] H. Chu, P. Gerhart, T. Ruffing, and D. Schröder. “Practical Schnorr Threshold Signatures Without the Algebraic Group Model”. In: *CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023*. Ed. by H. Handschuh and A. Lysyanskaya. Vol. 14081. LNCS. Springer, 2023, pp. 743–773. DOI: 10.1007/978-3-031-38557-5_24. URL: https://doi.org/10.1007/978-3-031-38557-5_24.
 - [30] D. Connolly, C. Komlo, I. Goldberg, and C. Wood. *Two-Round Threshold Schnorr Signatures with FROST*. 2022. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-frost/>.
 - [31] E. Crites, J. Katz, C. Komlo, S. Tessaro, and C. Zhu. *On the Adaptive Security of FROST*. To appear in *CRYPTO 2025*.
 - [32] E. Crites, C. Komlo, and M. Maller. *On the Adaptive Security of Key-Unique Threshold Signatures*. Cryptology ePrint Archive, Paper 2025/943. 2025. URL: <https://eprint.iacr.org/2025/943>.
 - [33] E. Crites and A. Stewart. *A Plausible Attack on the Adaptive Security of Threshold Schnorr Signatures*. To appear in *CRYPTO 2025*.
 - [34] E. C. Crites, C. Komlo, and M. Maller. “Fully Adaptive Schnorr Threshold Signatures”. In: *CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023*. Ed. by H. Handschuh and A. Lysyanskaya. Vol. 14081. LNCS. Springer, 2023, pp. 678–709. DOI: 10.1007/978-3-031-38557-5_22. URL: https://doi.org/10.1007/978-3-031-38557-5_22.
 - [35] E. C. Crites, C. Komlo, and M. Maller. “How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures”. In: *IACR Cryptol. ePrint Arch.* (2021), p. 1375. URL: <https://eprint.iacr.org/2021/1375>.
 - [36] S. Das and L. Ren. “Adaptively Secure BLS Threshold Signatures from DDH and co-CDH”. In: *IACR Cryptol. ePrint Arch.* (2023), p. 1553. URL: <https://eprint.iacr.org/2023/1553>.
 - [37] M. Drijvers et al. “On the Security of Two-Round Multi-Signatures”. In: *SP 2019, San Francisco, CA, USA, May 19-23, 2019*. IEEE, 2019, pp. 1084–1101.
 - [38] B. Edgington. *Upgrading Ethereum*. 2023. URL: https://eth2book.info/bellatrix/part2/building_blocks/randomness/.
 - [39] A. Fiat and A. Shamir. “How to Prove Yourself: Practical Solutions to Identification and Signature Problems”. In: *CRYPTO 1986, Santa Barbara, California, USA, 1986*. Ed. by A. M. Odlyzko. Vol. 263. LNCS. Springer, 1986, pp. 186–194.
 - [40] M. Fischlin. “Communication-Efficient Non-interactive Proofs of Knowledge with Online Extractors”. In: *CRYPTO 2005, Santa Barbara, California, USA, August 14-18, 2005*. Ed. by V. Shoup. Vol. 3621. LNCS. Springer, 2005, pp. 152–168.

- [41] Y. Frankel, P. D. MacKenzie, and M. Yung. “Adaptively-Secure Distributed Public-Key Systems”. In: *Algorithms - ESA '99, 7th Annual European Symposium, Prague, Czech Republic, July 16-18, 1999, Proceedings*. Ed. by J. Nešetřil. Vol. 1643. Lecture Notes in Computer Science. Springer, 1999, pp. 4–27. DOI: 10.1007/3-540-48481-7_2. URL: https://doi.org/10.1007/3-540-48481-7_2.
- [42] Y. Frankel, P. D. MacKenzie, and M. Yung. “Adaptively-Secure Optimal-Resilience Proactive RSA”. In: *Advances in Cryptology - ASIACRYPT '99, International Conference on the Theory and Applications of Cryptology and Information Security, Singapore, November 14-18, 1999, Proceedings*. Ed. by K. Lam, E. Okamoto, and C. Xing. Vol. 1716. Lecture Notes in Computer Science. Springer, 1999, pp. 180–194. DOI: 10.1007/978-3-540-48000-6_15. URL: https://doi.org/10.1007/978-3-540-48000-6_15.
- [43] G. Fuchsbauer, E. Kiltz, and J. Loss. “The Algebraic Group Model and its Applications”. In: *CRYPTO 2018, Santa Barbara, CA, USA, August 19-23, 2018*. Ed. by H. Shacham and A. Boldyreva. Vol. 10992. LNCS. Springer, 2018, pp. 33–62.
- [44] G. Fuchsbauer, A. Plouviez, and Y. Seurin. “Blind Schnorr Signatures and Signed ElGamal Encryption in the Algebraic Group Model”. In: *EUROCRYPT 2020, Zagreb, Croatia, May 10-14, 2020*. Ed. by A. Canteaut and Y. Ishai. Vol. 12106. LNCS. Springer, 2020, pp. 63–95.
- [45] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. “Secure Applications of Pedersen’s Distributed Key Generation Protocol”. In: *CT-RSA 2003, San Francisco, CA, USA, April 13-17, 2003*. Ed. by M. Joye. Vol. 2612. LNCS. Springer, 2003, pp. 373–390.
- [46] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin. “Secure Distributed Key Generation for Discrete-Log Based Cryptosystems”. In: *J. Cryptol.* 20.1 (2007), pp. 51–83.
- [47] C. Gentry and D. Wichs. “Separating succinct non-interactive arguments from all falsifiable assumptions”. In: *STOC 2011, San Jose, CA, USA, 6-8 June 2011*. Ed. by L. Fortnow and S. P. Vadhan. ACM, 2011, pp. 99–108.
- [48] S. Goldwasser, S. Micali, and R. L. Rivest. “A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks”. In: *SIAM J. Comput.* 17.2 (1988), pp. 281–308.
- [49] S. Jarecki and A. Lysyanskaya. “Adaptively Secure Threshold Cryptography: Introducing Concurrency, Removing Erasures”. In: *EUROCRYPT 2000, Bruges, Belgium, May 14-18, 2000*. Ed. by B. Preneel. Vol. 1807. LNCS. Springer, 2000, pp. 221–242.
- [50] S. Katsumata, M. Reichle, and K. Takemure. “Adaptively Secure 5 Round Threshold Signatures from sfMLWE /sfMSIS and sfDL with Rewinding”. In: *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*. Ed. by L. Reyzin and D. Stebila. Vol. 14926. Lecture Notes in Computer Science. Springer, 2024, pp. 459–491. DOI: 10.1007/978-3-031-68394-7_15. URL: https://doi.org/10.1007/978-3-031-68394-7_15.
- [51] N. Kobitz and A. Menezes. “Another look at non-standard discrete log and Diffie-Hellman problems”. In: *J. Math. Cryptol.* 2.4 (2008), pp. 311–326.
- [52] C. Komlo and I. Goldberg. “FROST: Flexible Round-Optimized Schnorr Threshold Signatures”. In: *SAC 2020, Halifax, NS, Canada (Virtual Event), October 21-23, 2020*. Ed. by O. Dunkelman, M. J. J. Jr., and C. O’Flynn. Vol. 12804. LNCS. Springer, 2020, pp. 34–65. DOI: 10.1007/978-3-030-81652-0_2.

- [53] B. Libert, M. Joye, and M. Yung. “Born and raised distributively: Fully distributed non-interactive adaptively-secure threshold signatures with short shares”. In: *Theoretical Computer Science* 645 (2016), pp. 1–24.
- [54] Y. Lindell. “Simple Three-Round Multiparty Schnorr Signing with Full Simulatability”. In: *IACR Cryptol. ePrint Arch.* (2022), p. 374. URL: <https://eprint.iacr.org/2022/374>.
- [55] A. Lysyanskaya and C. Peikert. “Adaptive Security in the Threshold Setting: From Cryptosystems to Signature Schemes”. In: *ASIACRYPT 2001, Gold Coast, Australia, December 9-13, 2001*. Ed. by C. Boyd. Vol. 2248. LNCS. Springer, 2001, pp. 331–350.
- [56] N. Makriyannis. *On the Classic Protocol for MPC Schnorr Signatures*. Cryptology ePrint Archive, Paper 2022/1332. <https://eprint.iacr.org/2022/1332>. 2022. URL: <https://eprint.iacr.org/2022/1332>.
- [57] G. Maxwell, A. Poelstra, Y. Seurin, and P. Wuille. “Simple Schnorr multi-signatures with applications to Bitcoin”. In: *Des. Codes Cryptogr.* 87.9 (2019), pp. 2139–2164.
- [58] J. Nick, T. Ruffing, and Y. Seurin. “MuSig2: Simple Two-Round Schnorr Multi-signatures”. In: *CRYPTO 2021, Virtual Event, August 16-20, 2021*. Ed. by T. Malkin and C. Peikert. Vol. 12825. LNCS. Springer, 2021, pp. 189–221.
- [59] A. Nicolosi, M. N. Krohn, Y. Dodis, and D. Mazières. “Proactive Two-Party Signatures for User Authentication”. In: *Proceedings of the Network and Distributed System Security Symposium, NDSS 2003, San Diego, California, USA*. The Internet Society, 2003. URL: <https://www.ndss-symposium.org/ndss2003/proactive-two-party-signatures-user-authentication/>.
- [60] D. Pointcheval and J. Stern. “Security Arguments for Digital Signatures and Blind Signatures”. In: *J. Cryptol.* 13.3 (2000), pp. 361–396.
- [61] D. Pointcheval and J. Stern. “Security Proofs for Signature Schemes”. In: *EUROCRYPT 1996, Saragossa, Spain, May 12-16, 1996*. Ed. by U. M. Maurer. Vol. 1070. LNCS. Springer, 1996, pp. 387–398.
- [62] T. Ruffing, V. Ronge, E. Jin, J. Schneider-Bensch, and D. Schröder. “ROAST: Robust Asynchronous Schnorr Threshold Signatures”. In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*. Ed. by H. Yin, A. Stavrou, C. Cremers, and E. Shi. ACM, 2022, pp. 2551–2564. DOI: 10.1145/3548606.3560583. URL: <https://doi.org/10.1145/3548606.3560583>.
- [63] C. Schnorr. “Efficient Signature Generation by Smart Cards”. In: *J. Cryptol.* 4.3 (1991), pp. 161–174.
- [64] A. Shamir. “How to Share a Secret”. In: *Commun. ACM* 22.11 (1979), pp. 612–613.
- [65] D. R. Stinson and R. Stroh. “Provably Secure Distributed Schnorr Signatures and a (t, n) Threshold Scheme for Implicit Certificates”. In: *ACISP 2001, Sydney, Australia, July 11-13, 2001*. Ed. by V. Varadharajan and Y. Mu. Vol. 2119. LNCS. Springer, 2001, pp. 417–434.

A Proof of Static Security

We now prove Theorem 1. Our static security proof serves as a template for the adaptive security proofs later on. In particular, it uses the same simulation techniques, even if they are not strictly necessary in the static setting. In this

Algorithm $\text{Fork}^{\mathcal{C}}(X)$ Pick random coins ρ for $\mathcal{C}$. $h_1, \dots, h_q \leftarrow_{\\$} H$ $(I, \text{out}) \leftarrow \mathcal{C}(X, (h_1, \dots, h_q); \rho)$ return $\perp$ if $I = 0$ $h'_1, \dots, h'_q \leftarrow_{\\$} H$ $(I', \text{out}') \leftarrow \mathcal{C}(X, (h_1, \dots, h_{I-1}, h'_1, \dots, h'_q); \rho)$ return $\perp$ if $I \neq I' \vee h_I = h'_I$ return $\text{output} \leftarrow (I, \text{out}, \text{out}')$
--

Fig. 9. The forking algorithm $\text{Fork}^{\mathcal{C}}(X)$ associated to an algorithm $\mathcal{C}$ and input X .

way, static security can be viewed as a special case of adaptive security where all corrupt parties are corrupted at the onset of the protocol. This approach limits redundancy in the three proofs we present in this work and modularizes changes made in subsequent games when adaptive corruptions are added.

Our static and $t/2$ -adaptive security reductions make use of rewinding. We recall the general forking lemma by Bellare and Neven [12], which is a formalization of the forking lemma introduced by Pointcheval and Stern [61].

Lemma 1 (General Forking Lemma [12]). *Let $q \in \mathbb{N}$ and H be a finite, non-empty set. Let IG be a randomized algorithm that generates an input X . Let $\mathcal{C}$ be a randomized algorithm that takes as input X and $h_1, \dots, h_q \in H$ and random coins ρ and outputs an index $I \in [0..q]$ and side output out . Let $\text{accept}(\mathcal{C})$ be the probability that $\mathcal{C}$ outputs an index $I \neq 0$ in the following experiment:*

$$\text{accept}(\mathcal{C}) := \Pr[X \leftarrow_{\$} \text{IG}; h_1, \dots, h_q \leftarrow_{\$} H; (I, \text{out}) \leftarrow \mathcal{C}(X, (h_1, \dots, h_q); \rho) : I \neq 0]$$

Let the forking algorithm $\text{Fork}^{\mathcal{C}}(X)$ associated to $\mathcal{C}$ be the randomized algorithm that takes as input X and proceeds as in Figure 9. Let

$$\text{accept}(\text{Fork}^{\mathcal{C}}) := \Pr[X \leftarrow_{\$} \text{IG}; \text{output} \leftarrow_{\$} \text{Fork}^{\mathcal{C}}(X) : \text{output} \neq \perp]$$

Then, $\text{accept}(\text{Fork}^{\mathcal{C}})$ is bounded by

$$\text{accept}(\text{Fork}^{\mathcal{C}}) \geq \text{accept}(\mathcal{C}) \cdot \left(\frac{\text{accept}(\mathcal{C})}{q} - \frac{1}{|H|} \right). \quad (6)$$

or, alternatively,

$$\text{accept}(\mathcal{C}) \leq \frac{q}{|H|} + \sqrt{q \cdot \text{accept}(\text{Fork}^{\mathcal{C}})} \quad (7)$$

Proof. (of Theorem 1.) We prove the theorem by a sequence of games.

Game 0. This is the static unforgeability game as defined in Figure 7, instantiated with **Sparkle+**.

Let $\mathcal{A}$ be a PPT adversary attempting to break the static unforgeability of **Sparkle+** (Fig. 7) that makes up to q_H queries to H_{cm} and H_{sig} , and q_S queries to its signing oracles, each time it is executed. We assume, without loss of generality, that $\mathcal{A}$ makes a query to H_{sig} on its forgery (pk, m^*, R^*) . Then, let $q = q_H + q_S + 1$.

Let W_0 be the event that $\mathcal{A}$ wins Game 0. Then, the advantage of $\mathcal{A}$ is:

$$\text{Adv}_{\mathcal{A}, \text{Sparkle}^+}^{\text{TS-EUF-CMA}}(\kappa, \text{frac} = 1) = |\Pr[W_0]| \quad (8)$$

Here, $\text{frac} = 1$ to allow $\mathcal{A}$ to corrupt a full $\text{frac} \cdot t = t$ parties, which must be declared by $\mathcal{A}$ at the beginning of its execution.

Game 1. In this game, we rule out the event that $\mathcal{A}$ forges a DS signature for some honest party. That is, the game aborts if the following event **ForgeDS** occurs.

In Game 1, the environment initializes a table $\mathbf{Q}_{\text{msg}} \leftarrow \emptyset$. When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_2}$ on inputs $(k, \text{eid}, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$, the environment first performs the checks in $\mathcal{O}^{\text{Sign}_2}$ (Fig. 7) and Sign_2 (Fig. 8). If they pass, then, in particular, $\text{st}_{k, \text{eid}, 1} = (\text{cm}_k, R_k, r_k, \text{sk}_k)$, where $\text{cm}_k \in \{\text{cm}_i\}_{i \in \mathcal{S}}$. The environment can then form the message $\text{msg}_k = (k, \text{cm}_k, R_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$ and complete Round 2 honestly. Finally, the environment adds msg_k to $\mathbf{Q}_{\text{msg}}$.

When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_3}$ on inputs $(k, \text{eid}, \{(R_i, \hat{\sigma}_i)\}_{i \in \mathcal{S}})$, the environment first performs the checks in $\mathcal{O}^{\text{Sign}_3}$ (Fig. 7) and Sign_3 (Fig. 8). If they pass, then, in particular, $\text{st}_{k, \text{eid}, 2} = (R_k, r_k, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \text{sk}_k)$, where $(R_k, \hat{\sigma}_k) \in \{(R_i, \hat{\sigma}_i)\}_{i \in \mathcal{S}}$. The environment forms, for all $i \in \mathcal{S}$, the messages $\text{msg}_i = (i, \text{cm}_i, R_i, \mathcal{S}, m, \{\text{cm}_j\}_{j \in \mathcal{S}})$. If the following holds, the environment aborts:

$$\forall i \in \text{hon} \cap \mathcal{S}, \text{DS.Verify}(\hat{X}_i, \text{msg}_i, \hat{\sigma}_i) = 1, \text{ but } \exists i' \text{ s.t. } \text{msg}_{i'} \notin \mathbf{Q}_{\text{msg}} \quad (9)$$

That is, $\mathcal{A}$ has forged a signature $\hat{\sigma}_{i'}$ on a message $\text{msg}_{i'}$ that honest party i' never signed. We refer to this abort event as **ForgeDS**.

Reduction $\mathcal{B}_1$ to the EUF-CMA Security of DS. The event that **ForgeDS** occurs implies $\mathcal{A}$ breaks EUF-CMA security of DS (Fig. 4) for some public key $\hat{\text{pk}}_{i'}$. Therefore, we can define a reduction $\mathcal{B}_1$ against the EUF-CMA game. We show that when $\mathcal{A}$ wins Game 1, then $\mathcal{B}_1$ wins its game with non-negligible probability.

The reduction $\mathcal{B}_1^{\mathcal{O}^{\text{DS}, \text{Sign}}}(\text{par}_{\text{DS}}, \hat{X}^\dagger)$ accepts as input public parameters par_{DS} and a challenge public key $\hat{\text{pk}} \leftarrow \hat{X}^\dagger$; its goal is to output a valid forgery $(\text{msg}^*, \hat{\sigma}^*)$ under $\hat{\text{pk}}$ such that $\mathcal{B}_1$ has not queried msg^* to its signing oracle $\mathcal{O}^{\text{DS}, \text{Sign}}$.

Setup. $\mathcal{B}_1$ initializes the following values:

- $(\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\kappa)$ and parameters for DS by performing $\text{par}_{\text{DS}} \leftarrow \text{DS.Setup}(1^\kappa)$. $\mathcal{B}_1$ then sets $\text{par} \leftarrow ((\mathbb{G}, p, g), \text{par}_{\text{DS}})$.
- The set $\mathbf{Q}_{\text{msg}} \leftarrow \emptyset$ of messages msg_k signed by honest parties during $\mathcal{O}^{\text{Sign}_2}$.
- The set $\mathbf{Q}_m \leftarrow \emptyset$ of messages queried by $\mathcal{A}$ to $\mathcal{O}^{\text{Sign}_2}$.
- The set $\mathbf{Q}_{\text{st}} \leftarrow \emptyset$ of participants' states for all signing sessions initiated by $\mathcal{A}$.

$\mathcal{B}_1$ runs $(n, t+1, \text{cor}, \text{st}_{\mathcal{A}}) \leftarrow \mathcal{A}(\text{par})$, and sets $\text{hon} \leftarrow [n] \setminus \text{cor}$. $\mathcal{A}$ is given access to random oracles $\text{H}_{\text{cm}}, \text{H}_{\text{sig}}$.

$\mathcal{B}_1$ runs key generation for Sparkle+ honestly: $(\text{pk}, \{(\text{pk}_i, \text{sk}_i)\}_{i \in [n]}) \leftarrow \text{KeyGen}(n, t+1)$. Then, it samples $\tau \leftarrow \text{hon}$ and parses $(X_\tau, \hat{X}_\tau) \leftarrow \text{pk}_\tau$ and $(x_\tau, \hat{x}_\tau, \text{pk}, \{\text{pk}_i\}_{i \in [n]}) \leftarrow \text{sk}_\tau$. It replaces $\hat{X}_\tau$ with its challenge: $\text{pk}_\tau \leftarrow (X_\tau, \hat{X}_\tau^\dagger)$ and $\text{sk}_\tau \leftarrow (x_\tau, \perp, \text{pk}, \{\text{pk}_i\}_{i \in [n]})$.

$\mathcal{B}_1$ then runs $\mathcal{O}^{\text{Sign}_1, \text{Sign}_2, \text{Sign}_3}(\text{pk}, \{\text{pk}_i\}_{i \in [n]}, \{\text{sk}_j\}_{j \in \text{cor}}, \text{st}_{\mathcal{A}})$.

Simulating Random Oracle Queries. When $\mathcal{A}$ queries H_{cm} or H_{sig} , $\mathcal{B}_1$ follows the protocol honestly.

Simulating Signing Oracle Queries. When $\mathcal{A}$ queries its signing oracles, $\mathcal{B}_1$ does as follows:

- $\mathcal{O}^{\text{Sign}_1}$: When $\mathcal{A}$ queries on $k \in \text{hon}$, $\mathcal{B}_1$ follows the protocol honestly.
- $\mathcal{O}^{\text{Sign}_2}$: When $\mathcal{A}$ queries on $(k, \text{eid}, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$, $\mathcal{B}_1$ first performs the checks in $\mathcal{O}^{\text{Sign}_2}$ (Fig. 7) and Sign_2 (Fig. 8). If they pass, then $\mathcal{B}_1$ updates $\text{Q}_m \leftarrow \text{Q}_m \cup \{m\}$, and computes $\text{msg}_k = (k, \text{cm}_k, R_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$. It then does the following:
 1. If $k \neq \tau$, then $\mathcal{B}_1$ follows the protocol honestly and computes the DS signature $\hat{\sigma}_k$ on msg_k .
 2. If $k = \tau$, then $\mathcal{B}_1$ queries its own signing oracle $\mathcal{O}^{\text{DS.Sig}}(\text{msg}_\tau)$ to obtain $\hat{\sigma}_\tau$. It otherwise follows the protocol honestly.
 Finally, $\mathcal{B}_1$ updates $\text{Q}_{\text{msg}} \leftarrow \text{Q}_{\text{msg}} \cup \{\text{msg}_k\}$, indicating that honest party k output a signature $\hat{\sigma}_k$ on msg_k .
- $\mathcal{O}^{\text{Sign}_3}$: When $\mathcal{A}$ queries on $(k, \text{eid}, \{(R_i, \hat{\sigma}_i)\}_{i \in \mathcal{S}})$, $\mathcal{B}_1$ first performs the checks in $\mathcal{O}^{\text{Sign}_3}$ (Fig. 7) and Sign_3 (Fig. 8). If they pass, then $\mathcal{B}_1$ checks if Condition 9 holds. If it does not hold, $\mathcal{B}_1$ follows the protocol honestly. Otherwise, if Condition 9 holds, then ForgeDS has occurred. If ForgeDS has occurred for $i' = \tau$ (i.e., $\mathcal{A}$ has output a forgery for the simulated player τ), then $\mathcal{B}_1$ terminates the execution of $\mathcal{A}$ and returns $(\text{msg}_\tau, \hat{\sigma}_\tau)$ as its forgery for the EUF-CMA game. Otherwise, $\mathcal{B}_1$ aborts.

If $\mathcal{A}$ terminates and ForgeDS does not occur, then $\mathcal{B}_1$ outputs 0 for the EUF-CMA game.

Analysis of $\mathcal{B}_1$'s Simulation. $\mathcal{B}_1$ simulates H_{cm} and H_{sig} perfectly because it follows the protocol honestly.

$\mathcal{B}_1$ responds to $\mathcal{O}^{\text{Sign}_1}$ queries honestly, so its simulation is perfect. It also responds to $\mathcal{O}^{\text{Sign}_2}$ and $\mathcal{O}^{\text{Sign}_3}$ queries honestly, with the exception of participant τ . However, because $\mathcal{B}_1$ queries its own EUF-CMA signing oracle $\mathcal{O}^{\text{DS.Sig}}$ to receive signatures that verify under $\text{pk}_\tau = \hat{X}_\tau^\dagger$, $\mathcal{B}_1$'s simulation is perfect.

Difference between Game 0 and Game 1. If ForgeDS does not occur, then Game 0 and Game 1 are identical. If ForgeDS occurs, then the probability that $\mathcal{A}$ forges for participant τ is at least $\frac{1}{n}$. The additional advantage for $\mathcal{A}$ in Game 1, assuming that ForgeDS occurs for participant τ , is upper-bounded by the advantage that $\mathcal{B}_1$ wins its EUF-CMA game against DS.

Let W_1 be the event that $\mathcal{A}$ wins Game 1. Then:

$$|\Pr[W_0] - \Pr[W_1]| \leq n \text{Adv}_{\mathcal{B}_1, \text{DS}}^{\text{EUF-CMA}}(\kappa) \quad (10)$$

Game 2. Next, we define a game that rules out several bad events: ROCollision , $\text{H}_{\text{sig}}\text{Guess}$, ForgeDS_k , ForgeDS_i , FromInvalid_k . That is, the game aborts if any of these events occurs.

We next define a simulator algorithm SIM that simulates Game 1 to a static unforgeability adversary $\mathcal{A}$.

Setup. SIM accepts as input an instance X , which is a tuple consisting of the group description $(\mathbb{G}, p, g)$ and a discrete logarithm (DL) challenge $\dot{X} \in \mathbb{G}$. In addition, SIM accepts as input a set of $q = q_H + q_S + 1$ values $\{h_1, \dots, h_q\}$, which it will use to program its random oracles responses (i.e., $\text{SIM} = \mathcal{C}$ in the forking lemma notation of Fig. 9).

Next, SIM checks for collisions $h_i = h_j$ for $i, j \in [q], i \neq j$. If so, SIM aborts. We refer to this bad event as ROCollision .

Otherwise, SIM initializes the following values:

- Parameters for DS by performing $\text{par}_{\text{DS}} \leftarrow \text{DS.Setup}(1^\kappa)$. SIM then sets $\text{par} \leftarrow ((\mathbb{G}, p, g), \text{par}_{\text{DS}})$.
- The sets $\text{Q}_{\text{cm}} \leftarrow \emptyset, \text{Q}_{\text{sig}} \leftarrow \emptyset$ of $\text{H}_{\text{cm}}, \text{H}_{\text{sig}}$ queries and their responses, respectively.
- The set $\text{Q}_{\text{msg}} \leftarrow \emptyset$ of messages msg_k signed by honest parties during $\mathcal{O}^{\text{Sign}_2}$.
- The set $\text{Q}_m \leftarrow \emptyset$ of messages queried by $\mathcal{A}$ to $\mathcal{O}^{\text{Sign}_2}$.
- The set $\text{Q}_{\text{st}} \leftarrow \emptyset$ of participants' states for all signing sessions initiated by $\mathcal{A}$.
- The set $\text{Q}_{\text{cm-wait}} \leftarrow \emptyset$ of potential Q_{cm} stored values.
- The counter $\text{ctr} \leftarrow 1$.

$\mathcal{A}$ is given access to random oracles $\text{H}_{\text{cm}}, \text{H}_{\text{sig}}$, which SIM may program.

Static Corruption. SIM then runs $\mathcal{A}(\text{par})$. $\mathcal{A}$ chooses the total number of potential signers n , the threshold $t + 1 \leq n$, and the set of corrupt parties $\text{cor} \leftarrow \{j\}, j \in [n], |\text{cor}| \leq t$, which are fixed for the rest of the protocol. If $t + 1 > n$ or $\text{cor} \not\subseteq [n]$ or $|\text{cor}| > t$, SIM outputs $\perp$. Otherwise, SIM sets $\text{hon} \leftarrow [n] \setminus \text{cor}$ and must reveal the secret keys of the corrupt parties to $\mathcal{A}$, which SIM does in the next step.

Simulating KeyGen. SIM simulates the key generation algorithm (Fig. 8) using its DL challenge $\dot{X}$ as follows.

1. SIM generates public and private DS keys honestly $(\hat{X}_i, \hat{x}_i) \leftarrow_s \text{DS.KeyGen}()$, for all $i \in [n]$.
2. SIM sets the threshold public key $\text{pk} \leftarrow \dot{X}$.
3. SIM simulates a Shamir secret sharing of the discrete logarithm of pk by performing the following steps. Assume without loss of generality that $|\text{cor}| = t$.
 - (a) SIM samples t random values $x_j \leftarrow_s \mathbb{Z}_p$ for $j \in \text{cor}$.

- (b) Let f be the polynomial whose constant term is the challenge $f(0) = \dot{x}$ (which SIM does not know) and for which $f(j) = x_j$ for all $j \in \text{cor}$. SIM computes the $t+1$ Lagrange polynomials $\{L'_0(Z), \{L'_j(Z)\}_{j \in \text{cor}}\}$ relating to the set (of x -coordinates) $0 \cup \text{cor}$. (See Section 3 for notation.)
- (c) For all $j \in \text{cor}$, SIM sets the public key share as $X_j \leftarrow g^{x_j}$. For all $i \in \text{hon}$, SIM computes:

$$X_i = \text{pk}^{L'_{0,i}} \prod_{j \in \text{cor}} g^{x_j L'_{j,i}}$$

where $L'_{j,i}$ is the i^{th} coefficient of $L'_j(Z) = L'_{j,0} + L'_{j,1}Z + \dots + L'_{j,t}Z^t$.

4. SIM sets $\text{pk}_i \leftarrow (X_i, \hat{X}_i)$ and $\text{sk}_i \leftarrow (x_i, \hat{x}_i)$ for all $i \in [n]$.

The threshold public key is $\text{pk} = g^{f(0)} = \dot{X}$ with corresponding secret key $\text{sk} = \dot{x}$. SIM then runs $\mathcal{A}^{\mathcal{O}^{\text{Sign}_1, \text{Sign}_2, \text{Sign}_3}}(\text{pk}, \{\text{pk}_i\}_{i \in [n]}, \{\text{sk}_j\}_{j \in \text{cor}}, \text{st}_{\mathcal{A}})$.

Simulating Random Oracle Queries. When $\mathcal{A}$ queries H_{cm} or H_{sig} , SIM simulates the response as follows.

H_{cm} : When $\mathcal{A}$ queries H_{cm} on (i, R_i) , SIM checks whether Q_{cm} contains an entry $(i, R_i, \text{cm}_i, \cdot) \in \text{Q}_{\text{cm}}$, and if so, it returns cm_i . (Note: $\cdot$ means a defined entry z_i , or an undefined entry $\perp$.)

Else, SIM sets $\text{cm}_i \leftarrow h_{\text{ctr}}$ and updates $\text{Q}_{\text{cm}} \leftarrow \text{Q}_{\text{cm}} \cup \{(i, R_i, \text{cm}_i, \perp)\}$. It then increments $\text{ctr} \leftarrow \text{ctr} + 1$ and returns cm_i .

H_{sig} : When $\mathcal{A}$ queries H_{sig} on (X, m, R) , SIM checks whether $(X, m, R, c, \Delta) \in \text{Q}_{\text{sig}}$ and, if so, returns c .

Else, SIM sets $c \leftarrow h_{\text{ctr}}$ and $\Delta \leftarrow \text{ctr}$. It appends (X, m, R, c, Δ) to Q_{sig} . It then increments $\text{ctr} \leftarrow \text{ctr} + 1$ and returns c .

Simulating Sparkle+ Signing. SIM handles $\mathcal{A}$'s signing queries as follows.

Round 1 ($\mathcal{O}^{\text{Sign}_1}(k)$): In the first round of signing, parties form commitments cm_i . When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_1}$ for participant identifier k , SIM first checks that $k \in \text{hon}$. If so, then SIM does the following:

1. Samples $\text{eid} \leftarrow_{\$} \{0, 1\}^*$.
2. Sets $\text{cm}_k \leftarrow h_{\text{ctr}}$, and increments $\text{ctr} \leftarrow \text{ctr} + 1$.
3. Sets $\text{st}_{k, \text{eid}, 1} \leftarrow (\text{cm}_k, \perp, \perp, \hat{\text{sk}}_k)$, where $\hat{\text{sk}}_k := (\perp, \hat{x}_k, \text{pk}, \{\text{pk}_i\}_{i \in [n]})$.
4. Updates $\text{Q}_{\text{st}} \leftarrow \text{Q}_{\text{st}}[k, \text{eid}, 1] \leftarrow \text{st}_{k, \text{eid}, 1}$.
5. Finally, SIM returns cm_k to $\mathcal{A}$.

Round 2 ($\mathcal{O}^{\text{Sign}_2}(k, \text{eid}, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$): In the second round of signing, each party i in the signing set $\mathcal{S}$ takes as input a set of commitments $\{\text{cm}_j\}_{j \in \mathcal{S}}$ and reveals its nonce R_i such that $\text{cm}_i = \text{H}_{\text{cm}}(i, R_i)$. Each party also signs their message $\text{msg}_i = (i, \text{cm}_i, R_i, \mathcal{S}, m, \{\text{cm}_j\}_{j \in \mathcal{S}})$ using DS.

When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_2}$ on $(k, \text{eid}, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$ for $k \in \text{hon}$, SIM first performs the checks in $\mathcal{O}^{\text{Sign}_2}$ (Fig. 7) and Sign_2 (Fig. 8). If they pass, then, in particular $\text{st}_{k, \text{eid}, 1} = (\text{cm}_k, \perp, \perp, \hat{\text{sk}}_k)$. SIM then does the following:

1. Updates $\text{Q}_m \leftarrow \text{Q}_m \cup \{m\}$.

2. Next, it checks if $(k, R_k, \text{cm}_k, z_k, (\mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})) \in Q_{\text{cm-wait}}$ for some R_k, z_k . If the check holds, SIM sets R_k accordingly. Then, SIM adds $(k, R_k, \text{cm}_k, z_k)$ to Q_{cm} and $\text{msg}_k = (k, \text{cm}_k, R_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$ to Q_{msg} , signs msg_k as $\hat{\sigma}_k \leftarrow \text{DS.Sign}(\hat{x}_k, \text{msg}_k)$, updates $\text{st}_{k,\text{eid},2} \leftarrow (R_k, \perp, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \hat{\text{sk}}_k)$, and returns $(R_k, \hat{\sigma}_k)$.
3. If the check fails, then for each commitment $\text{cm}_i \in \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}$, SIM checks if it has an entry $(i, R_i, \text{cm}_i, \cdot) \in Q_{\text{cm}}$.
 - (a) If there exists some $\text{cm}_i \in \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}$ such that $i \in \mathcal{S} \cap \text{hon}$ and cm_i has a corresponding R_i defined in Q_{cm} , then SIM goes to Case 1.
 - (b) If there exists some $\text{cm}_j \in \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}$ such that $j \in \mathcal{S} \cap \text{cor}$ and cm_j does not have a corresponding R_j defined in Q_{cm} , then SIM goes to Case 1.
 - (c) Otherwise, SIM goes to Case 2.

Case 1 (Invalid Query). In this case, SIM does the following:

1. Sample random $R_k \leftarrow \mathbb{G}$.
2. Program $Q_{\text{cm}} \leftarrow Q_{\text{cm}} \cup \{(k, R_k, \text{cm}_k, \perp)\}$.
3. Follow the protocol honestly to produce a signature $\hat{\sigma}_k$ on msg_k .
4. Add $Q_{\text{msg}} \leftarrow Q_{\text{msg}} \cup \{\text{msg}_k\}$, indicating that honest party k output a signature $\hat{\sigma}_k$ on msg_k .
5. Set $\text{st}_{k,\text{eid},2} \leftarrow (R_k, \perp, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \hat{\text{sk}}_k)$.
6. Update $Q_{\text{st}}[k, \text{eid}, 2] \leftarrow \text{st}_{k,\text{eid},2}$.
7. Return $(R_k, \hat{\sigma}_k)$.

Case 2 (Valid Query). In this case, SIM does the following:

1. Sets $c \leftarrow h_{\text{ctr}}$ and $\Delta \leftarrow \text{ctr}$ and increments $\text{ctr} \leftarrow \text{ctr} + 1$.
2. For all $i \in \mathcal{S} \cap \text{hon}$, SIM samples random $z_i \leftarrow \mathbb{Z}_p$ and computes $R_i \leftarrow g^{z_i} X_i^{-c\lambda_i}$, where λ_i is the Lagrange coefficient for party i in the set $\mathcal{S}$.
3. Next, SIM programs $\text{cm}_k \leftarrow H_{\text{cm}}(k, R_k)$. That is, SIM adds the entry $Q_{\text{cm}} \leftarrow Q_{\text{cm}} \cup \{(k, R_k, \text{cm}_k, z_k)\}$. Additionally, for all $i \in \mathcal{S} \cap \text{hon}$, $i \neq k$, SIM adds the entry $Q_{\text{cm-wait}} \leftarrow Q_{\text{cm-wait}} \cup \{(i, R_i, \text{cm}_i, z_i, (\mathcal{S}, m, \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}))\}$.
4. Next, for each $j \in \mathcal{S} \cap \text{cor}$, SIM obtains R_j by looking up $(j, R_j, \text{cm}_j, \perp) \in Q_{\text{cm}}$.
5. SIM then computes $R = \prod_{\ell \in \mathcal{S}} R_\ell$.
6. If there is a defined entry $(\text{pk}, m, R, c', \Delta') \in Q_{\text{sig}}$ for some c', Δ' , then SIM aborts. We refer to this bad event as $H_{\text{sig}}^{\text{Guess}}$.
7. Otherwise, SIM programs $c \leftarrow H_{\text{sig}}(\text{pk}, m, R)$. Specifically, SIM adds $Q_{\text{sig}} \leftarrow Q_{\text{sig}} \cup \{(\text{pk}, m, R, c, \Delta)\}$.
8. SIM follows the protocol honestly to produce a signature $\hat{\sigma}_k$ on msg_k and adds $Q_{\text{msg}} \leftarrow Q_{\text{msg}} \cup \{\text{msg}_k\}$, indicating that honest party k output a signature $\hat{\sigma}_k$ on msg_k .
9. It then sets $\text{st}_{k,\text{eid},2} \leftarrow (R_k, \perp, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \hat{\text{sk}}_k)$ and updates $Q_{\text{st}}[k, \text{eid}, 2] \leftarrow \text{st}_{k,\text{eid},2}$.
10. Finally, SIM returns $(R_k, \hat{\sigma}_k)$.

Round 3 ($\mathcal{O}^{\text{Sign}_3}(k, \text{eid}, \{(R_i, \hat{\sigma}_i)\}_{i \in \mathcal{S}})$): In the third round of signing, each party $i \in \mathcal{S}$ outputs the remaining part of its partial signature, namely z_i . When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_3}$ on $(k, \text{eid}, \{(R_i, \hat{\sigma}_i)\}_{i \in \mathcal{S}})$ for $k \in \text{hon}$, SIM first performs the checks in $\mathcal{O}^{\text{Sign}_3}$ (Fig. 7) and Sign_3 (Fig. 8). If they pass, then, in particular $\text{st}_{k, \text{eid}, 2} \leftarrow (R_k, \perp, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \hat{\text{sk}}_k)$. SIM then does the following:

1. Checks that $\text{DS.Verify}(\hat{X}_i, \text{msg}_i, \hat{\sigma}_i) = 1$ for all $i \in \mathcal{S}$, where $\text{msg}_i = (i, \text{cm}_i, R_i, \mathcal{S}, m, \{\text{cm}_\ell\}_{\ell \in \mathcal{S}})$, and returns $\perp$ if not.
2. Checks whether $\text{cm}_i = \text{H}_{\text{cm}}(i, R_i)$ for all $i \in \mathcal{S}$ and returns $\perp$ if not.
3. Looks up $\text{msg}_k = (k, \text{cm}_k, R_k, \mathcal{S}, m, \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}) \in \mathcal{Q}_{\text{msg}}$ and aborts if no such entry exists. We refer to this bad event as ForgeDS_k .
4. Check that there exists cm_i such that $\text{msg}_i = (i, \text{cm}_i, R_i, \mathcal{S}, m, \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}) \in \mathcal{Q}_{\text{msg}}$ for all $i \in \mathcal{S} \cap \text{hon}$ and abort if not. We refer to this bad event as ForgeDS_i .
5. Else, SIM retrieves $(k, R_k, \text{cm}_k, \cdot) \in \mathcal{Q}_{\text{cm}}$ and aborts if $\cdot = \perp$. We refer to this bad event as FromInvalid_k . Otherwise, SIM retrieves z_k from $(k, R_k, \text{cm}_k, z_k) \in \mathcal{Q}_{\text{cm}}$.
6. SIM updates $\mathcal{Q}_{\text{st}}[k, \text{eid}, 3] \leftarrow z_k$.
7. Finally, SIM returns z_k .

Analysis of SIM's Simulation. SIM's simulation of key generation and $\mathcal{O}^{\text{Sign}_1}$ is perfect and no abort conditions occur.

SIM's simulation of $\mathcal{O}^{\text{Sign}_2}$ is perfect. Before revealing an honest R_k , SIM always programs H_{cm} such that $\text{cm}_k = \text{H}_{\text{cm}}(k, R_k)$, as would be the case in Game 2. Each R_k is randomized by z_k . SIM responds if and only if $\text{st}_{k, \text{eid}, 1}$ exists as in Game 2. The signatures $\hat{\sigma}_k$ verify on msg_k .

$\text{H}_{\text{sig}}\text{Guess}$ occurs with maximum probability $\frac{q}{p}$. Indeed, SIM checks to see if there is a defined entry $(\text{pk}, m, R, c', \Delta') \in \mathcal{Q}_{\text{sig}}$ for some c', Δ' before attempting to program H_{sig} . The probability that $\mathcal{A}$ guesses R in some hash query is bounded by $\frac{q}{p}$ because R is randomized by c .

SIM's simulation of $\mathcal{O}^{\text{Sign}_3}$ is perfect. If there is a defined entry $(k, R_k, \text{cm}_k, z_k) \in \mathcal{Q}_{\text{cm}}$, then $R_k = g^{z_k} X_k^{-c\lambda_k}$ for $c = \text{H}_{\text{sig}}(\text{pk}, m, R)$. Note that the aggregate R is computed consistently if all honest R_i 's are computed in the same $\mathcal{O}^{\text{Sign}_2}$ query with the same $(m, \mathcal{S}, \{\text{cm}_\ell\}_{\ell \in \mathcal{S}})$ during Case 2 (Valid Query). Due to ForgeDS_i , SIM only needs to respond to such queries. Each corrupt R_j hashes to a unique cm_j .

If two (i, R_i) and (i', R'_i) 's hash to the same cm_i , then ROCollision has occurred.

ForgeDS_k and ForgeDS_i cannot occur unless Game 1 also would abort.

FromInvalid_k occurs in Round 3 if some honest R_k is computed in an $\mathcal{O}^{\text{Sign}_2}$ query during Case 1 (Invalid Query). If this occurs in Step 3 (a) of the simulator's Round 2, then there exists some $\text{cm}_i \in \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}$ such that $i \in \mathcal{S} \cap \text{hon}$ and cm_i has a corresponding R_i defined in $\mathcal{Q}_{\text{cm}}$. Then ForgeDS_k or ForgeDS_i will occur because either (i) cm_i was determined according to a different Round 2 query $(m', \mathcal{S}', \{\text{cm}'_i\}_{i \in \mathcal{S}'})$ to cm_k , or else (ii) $\mathcal{A}$ chose R_i and queried $\text{H}_{\text{cm}}(i, R_i)$ to get cm_i . If this occurs in Step 3 (b), then there exists some $\text{cm}_j \in \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}$ such that $j \in \mathcal{S} \cap \text{cor}$ and cm_j does not have a corresponding R_j defined in $\mathcal{Q}_{\text{cm}}$,

but an adversary later finds R_j such that $H_{\text{cm}}(j, R_j) = \text{cm}_j$. This occurs with maximum probability $\frac{q}{p}$.

In summary, SIM aborts with negligible probability in the following cases:

1. ROCollision: SIM aborts due to ROCollision with probability $\frac{q^2}{2p}$, which occurs when it receives any inputs $\{h_1, \dots, h_q\}$ that collide.
2. H_{sig}Guess: SIM aborts due to H_{sig}Guess with probability q/p .
3. ForgeDS_k: When SIM aborts due to ForgeDS_k, then Game 1 also aborts.
4. ForgeDS_i: When SIM aborts due to ForgeDS_i, then Game 1 also aborts.
5. FromInvalid_k: SIM aborts due to FromInvalid_k with probability q/p .

Output. When $\mathcal{A}$ terminates with output (m^*, σ^*) , SIM first checks that it is a valid forgery, by checking that $m^* \notin Q_m$ and $\text{Verify}(\text{pk}, m^*, \sigma^*) = 1$. If either check fails, SIM returns 0.

Otherwise, SIM looks up the entry $(\text{pk}, m^*, R^*, c^*, I) \in Q_{\text{sig}}$ corresponding to $\mathcal{A}$'s forgery. It sets $\text{out} \leftarrow \sigma^*$. SIM then outputs (I, out) .

Difference between Game 2 and Game 1. Let W_2 be the event that $\mathcal{A}$ wins Game 2. The only additional advantage in the simulation by SIM over Game 1 is given by bad events ROCollision, H_{sig}Guess, and FromInvalid_k. As such,

$$\begin{aligned} |\Pr[W_1] - \Pr[W_2]| &= \Pr[\text{ROCollision}] + \Pr[\text{H}_{\text{sig}}\text{Guess}] + \Pr[\text{FromInvalid}_k] \\ &= \frac{q^2}{2p} + \frac{2q}{p} \end{aligned} \quad (11)$$

Reduction $\mathcal{B}_2$ Against DL. Finally, we construct our PPT reduction $\mathcal{B}_2$ against the DL assumption (Fig. 5). Specifically, we show how $\mathcal{B}_2$ uses Fork^{SIM} as a subroutine such that:

$$\text{Adv}_{\mathcal{B}_2}^{\text{dl}}(\kappa) \geq \text{accept}(\text{Fork}^{\text{SIM}}(X)) - 2\Pr[\text{ROCollision}] - \dots - 2\Pr[\text{FromInvalid}_k] \quad (12)$$

where X is the instance given by the challenger in the DL game.

Initialization. $\mathcal{B}_2$ is initialized by the DL challenger with input the group description $(\mathbb{G}, p, g)$ and a DL challenge $\dot{X} \in \mathbb{G}$. The goal of $\mathcal{B}_2$ is to output $\dot{x}$ such that $\dot{X} = g^{\dot{x}}$.

Execution. $\mathcal{B}_2$ then runs the general forking algorithm $\text{Fork}^{\text{SIM}}(X)$ as described in Figure 9, on the simulator algorithm SIM and its instance $X = ((\mathbb{G}, p, g), \dot{X})$. With non-negligible probability lower-bounded by Equation 6, $\text{Fork}^{\text{SIM}}(X)$ will output the accepting answer $(I, \text{out}, \text{out}')$ such that:

$$\begin{aligned} h_I &= c^*, h'_I = c^{**}, \\ \sigma^* &\leftarrow \text{out}, \sigma^{**} \leftarrow \text{out}' \\ (R^*, z^*) &\leftarrow \sigma^*, (R^*, z^{**}) \leftarrow \sigma^{**} \end{aligned}$$

$\mathcal{B}_2$ then solves for $\text{sk} = \frac{z^* - z^{**}}{c^* - c^{**}}$, where $\text{sk} = \dot{x}$ and where the DL challenge is $\dot{X} = g^{\dot{x}}$. $\mathcal{B}_2$ then returns $\dot{x}$ as its output to the DL challenger.

Finishing the Proof. Equation 6 lower bounds $\text{accept}(\text{Fork}^{\text{SIM}}(X))$. The probability that $\mathcal{B}_2$ succeeds is therefore defined by combining Equations 6, 8, 10, 11, and 12, which gives Equation 3. This completes the proof. $\square$

B Proof of Adaptive Security up to $t/2$ Corruptions

In this section, we prove the adaptive security of **Sparkle+** against up to $t/2$ corruptions under the algebraic one-more discrete logarithm assumption (AOMDL) (Fig. 5) in the random oracle model (ROM). The reason the allowed corruption is $t/2$ is that, in order to extract an AOMDL solution, the reduction needs to rewind the adversary once, and there is no guarantee that the adversary will corrupt the same set of parties after the fork as it did during the first iteration of the adversary. When the adversary can corrupt only $t/2$ parties, this causes no issues, as the total number of corruptions over both iterations does not exceed t . If the adversary could corrupt more parties, then the reduction would query its discrete logarithm oracle more than t times and would lose the $t + 1\text{-aomdl}$ game.

Proof. (of Theorem 2.) We prove the theorem by a sequence of games.

Game 0. This is the adaptive unforgeability game as defined in Figure 7, instantiated with **Sparkle+**.

Let $\mathcal{A}$ be a PPT adversary attempting to break the adaptive unforgeability of **Sparkle+** (Fig. 7) that makes up to q_H queries to H_{cm} and H_{sig} , and q_S queries to its signing oracles, each time it is executed. $\mathcal{A}$ is allowed to make $t/2$ adaptive corruptions. We assume, without loss of generality, that $\mathcal{A}$ makes a query to H_{sig} on its forgery (pk, m^*, R^*) . Then, let $q = q_H + q_S + 1$.

Let W_0 be the event that $\mathcal{A}$ wins Game 0. Then, the advantage of $\mathcal{A}$ is:

$$\text{Adv}_{\mathcal{A}, \text{Sparkle}^+}^{\text{adp-TS-EUF-CMA}}(\kappa, \text{frac} = 1/2) = |\Pr[W_0]| \quad (13)$$

Here, $\text{frac} = 1/2$ to allow $\mathcal{A}$ to corrupt a $\text{frac} \cdot t = t/2$ parties.

Game 1. The changes from Game 0 to Game 1 are the same as in the proof of Theorem 1.

Reduction $\mathcal{B}'_1$ to the EUF-CMA Security of DS. We now define a reduction $\mathcal{B}'_1$ against the EUF-CMA security of DS that uses $\mathcal{A}$ as a subroutine. We show that when $\mathcal{A}$ wins Game 1, then $\mathcal{B}'_1$ wins the EUF-CMA game with non-negligible probability.

The only difference in $\mathcal{B}'_1$'s simulation to the reduction $\mathcal{B}_1$ in Game 1 for Theorem 1 is that $\mathcal{B}'_1$ must additionally respond to corruption queries $\mathcal{O}^{\text{Corrupt}}$.

Simulating Corruption Oracle Queries. When $\mathcal{A}$ queries its corruption oracle on $k \in \text{hon}$, $\mathcal{B}'_1$ does as follows:

1. If $k \neq \tau$, then $\mathcal{B}'_1$ follows the protocol honestly.
2. If $k = \tau$, then $\mathcal{B}'_1$ aborts. We refer to this abort event as **CorruptParty τ** .

Analysis of $\mathcal{B}'_1$'s Simulation. The only difference between $\mathcal{B}_1$'s simulation in Game 1 of Theorem 1 and $\mathcal{B}'_1$'s simulation is the occurrence of **CorruptParty τ** . At the end of the game, there are $(n - t/2)$ non-corrupted parties. The probability that τ is one of them (i.e., that **CorruptParty τ** does not occur) is $\frac{(n-t/2)}{n}$.

Difference between Game 0 and Game 1. If **ForgeDS** does not occur, then Game 0 and Game 1 are identical. If **ForgeDS** occurs, then the probability that $k = \tau$ is at least $\frac{1}{n}$. The probability that **CorruptParty τ** does not occur, i.e., τ is not corrupted, is greater than $\frac{(n-t/2)}{n}$.

The additional advantage for $\mathcal{A}$ in Game 1, assuming that **ForgeDS** occurs for participant τ , is upper-bounded by the advantage that $\mathcal{B}'_1$ wins its EUF-CMA game against DS, and the probability that **CorruptParty τ** does not occur.

Let W_1 be the event that $\mathcal{A}$ wins Game 1. Then:

$$|\Pr[W_0] - \Pr[W_1]| \leq \frac{n^2}{(n - t/2)} \text{Adv}_{\mathcal{B}'_1, \text{DS}}^{\text{EUF-CMA}}(\kappa) \quad (14)$$

Game 2. Next, we define a game that rules out several bad events: **ROCollision**, **H_{sig}Guess**, **ForgeDS_k**, **ForgeDS_i**, **FromInvalid_k**, **H_{sig}Guess_{OneHon}**, **H_{sig}Guess_{LastHonCor}**, **H_{cm}Guess**. That is, the game aborts if any of these events occurs.

We next define a simulator algorithm **SIM'** that simulates Game 1 to an adaptive unforgeability adversary $\mathcal{A}$. The simulation by **SIM'** is similar to that of **SIM** in Game 2 for Theorem 1, with the exception of the instance X that **SIM'** accepts as input, how key generation is performed, what information **SIM'** stores in $\mathcal{Q}_{\text{cm}}$, and how **SIM'** additionally simulates $\mathcal{O}^{\text{Corrupt}}$.

Additionally, **SIM'** is given an oracle $\mathcal{O}^{\text{dl}}$, which it can query on a group element, together with an algebraic representation, and receive its discrete logarithm. **SIM'** can query this oracle up to t times.

Setup. **SIM'** accepts as input an instance X , which is a tuple consisting of the group description $(\mathbb{G}, p, g)$ and $t + 1$ AOMDL challenges $(Y_0, \dots, Y_t) \in \mathbb{G}^{t+1}$. In addition, **SIM'** accepts as input a set of $q = q_H + q_S + 1$ values $\{h_1, \dots, h_q\}$, which it will use to program its random oracles responses.

Next, **SIM'** checks for collisions $h_i = h_j$ for $i, j \in [q], i \neq j$. If so, **SIM'** aborts. We refer to this bad event as **ROCollision**.

Otherwise, **SIM'** initializes the following values:

- Parameters for DS by performing $\text{par}_{\text{DS}} \leftarrow \text{DS.Setup}(1^\kappa)$. **SIM** then sets $\text{par} \leftarrow ((\mathbb{G}, p, g), \text{par}_{\text{DS}})$.
- The sets $\mathcal{Q}_{\text{cm}} \leftarrow \emptyset, \mathcal{Q}_{\text{sig}} \leftarrow \emptyset$ of $\text{H}_{\text{cm}}, \text{H}_{\text{sig}}$ queries and their responses, respectively.
- The set $\mathcal{Q}_{\text{msg}} \leftarrow \emptyset$ of messages msg_k signed by honest parties during $\mathcal{O}^{\text{Sign}_2}$.
- The set $\mathcal{Q}_m \leftarrow \emptyset$ of messages queried by $\mathcal{A}$ to $\mathcal{O}^{\text{Sign}_2}$.

- The set $Q_{\text{st}} \leftarrow \emptyset$ to be the participants' states for all signing sessions initiated by $\mathcal{A}$.
- The set $Q_{\text{cm-wait}} \leftarrow \emptyset$ of potential Q_{cm} and Q_{sig} stored values.
- The counter $\text{ctr} \leftarrow 1$.

$\mathcal{A}$ is given access to random oracles $H_{\text{cm}}, H_{\text{sig}}$, which SIM' may program.

Static Corruption. SIM' then runs $\mathcal{A}(\text{par})$. $\mathcal{A}$ chooses the total number of potential signers n and the threshold $t + 1 \leq n$. Additionally, $\mathcal{A}$ may choose a set of parties to corrupt initially: $\text{cor} \leftarrow \{j\}, j \in [n], |\text{cor}| \leq t/2$. If $t + 1 > n$ or $\text{cor} \not\subseteq [n]$ or $|\text{cor}| > t/2$, SIM' outputs $\perp$. Otherwise, SIM' sets $\text{hon} \leftarrow [n] \setminus \text{cor}$ and must reveal the secret keys of the corrupt parties to $\mathcal{A}$, which SIM' does in the next step.

Simulating KeyGen. SIM' simulates the key generation algorithm (Fig. 8) using its AOMDL challenge $(Y_0, \dots, Y_t)$ as follows.

1. SIM' generates public and private DS keys honestly $(\hat{X}_i, \hat{x}_i) \leftarrow_{\$} \text{DS.KeyGen}()$, for all $i \in [n]$.
2. For all $i \in [n]$, SIM' sets the public key share as:

$$X_i = Y_0 Y_1^i \cdots Y_t^i$$

which is implicitly equal to $g^{f(i)}$ for the polynomial $f(Z) = y_0 + y_1 Z + \cdots + y_t Z^t$. The threshold public key is $\text{pk} = g^{f(0)} = Y_0$ with corresponding secret key $\text{sk} = y_0$.

3. SIM' obtains the initial corrupt secret key shares by querying $x_j = f(j) \leftarrow \mathcal{O}^{\text{dl}}(X_j)$ (with representation $(1, j, \dots, j^t)$) for all $j \in \text{cor}$.
4. SIM' sets $\text{pk}_i \leftarrow (X_i, \hat{X}_i)$ and $\text{sk}_i \leftarrow (x_i, \hat{x}_i)$ for all $i \in [n]$.

SIM' then runs $\mathcal{A}^{\mathcal{O}^{\text{Sign}_1, \text{Sign}_2, \text{Sign}_3, \text{Corrupt}}}(\text{pk}, \{\text{pk}_i\}_{i \in [n]}, \{\text{sk}_j\}_{j \in \text{cor}}, \text{st}_{\mathcal{A}})$.

Simulating Random Oracle Queries. When $\mathcal{A}$ queries H_{cm} or H_{sig} , SIM' simulates the response as follows.

H_{cm} : When $\mathcal{A}$ queries H_{cm} on (i, R_i) , SIM' checks whether Q_{cm} contains an entry $(i, R_i, \cdot, \text{cm}_i, \cdot, \cdot) \in Q_{\text{cm}}$, and if so, it returns cm_i .

Else, SIM' sets $\text{cm}_i \leftarrow h_{\text{ctr}}$ and updates $Q_{\text{cm}} \leftarrow Q_{\text{cm}} \cup \{(i, R_i, \perp, \text{cm}_i, \perp, \perp)\}$. It then increments $\text{ctr} \leftarrow \text{ctr} + 1$ and returns cm_i .

H_{sig} : SIM' simulates $\mathcal{A}$'s queries to H_{sig} identically to SIM for Theorem 1.

Simulating Sparkle+ Signing. SIM' handles $\mathcal{A}$'s signing queries as follows.

Round 1 ($\mathcal{O}^{\text{Sign}_1}(k)$): SIM' simulates $\mathcal{A}$'s queries to $\mathcal{O}^{\text{Sign}_1}$ identically to SIM for Theorem 1, setting additional empty entries in Q_{cm} as required.

Round 2 ($\mathcal{O}^{\text{Sign}_2}(k, \text{eid}, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$): When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_2}$ on $(k, \text{eid}, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$ for $k \in \text{hon}$, SIM' performs the same steps as SIM for Theorem 1, with the following exceptions:

1. Updates $Q_m \leftarrow Q_m \cup \{m\}$.
2. Next, it checks if $(k, R_k, \text{cm}_k, z_k, (\mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}), c_k, c) \in Q_{\text{cm-wait}}$ for some R_k, z_k, c_k, c . If the check holds, SIM' sets R_k accordingly and:
 - (a) Adds $(k, R_k, \perp, \text{cm}_k, c_k, z_k)$ to Q_{cm} and $\text{msg}_k = (k, \text{cm}_k, R_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$ to Q_{msg} , signs msg_k as $\hat{\sigma}_k \leftarrow \text{DS.Sign}(\hat{x}_k, \text{msg}_k)$, and updates $\text{st}_{k,\text{eid},2} \leftarrow (R_k, \perp, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \hat{\text{sk}}_k)$.
 - (b) Checks if there exists $\{R_i\}_{i \in \mathcal{S}, i \neq k}$ such that $(i, R_i, \cdot, \text{cm}_i, \cdot, \cdot) \in Q_{\text{cm}}$ for all $i \in \mathcal{S}, i \neq k$. If the check holds, then SIM' computes $R = \prod_{i \in \mathcal{S}} R_i$. If $(\text{pk}, m, R, c', \Delta') \in Q_{\text{sig}}$ for some c', Δ' , then SIM' aborts. We refer to this bad event as $H_{\text{sig}}^{\text{Guess}}$. Otherwise, SIM' programs $c \leftarrow H_{\text{sig}}(\text{pk}, m, R)$. That is, SIM' adds $Q_{\text{sig}} \leftarrow Q_{\text{sig}} \cup \{(\text{pk}, m, R, c, \Delta)\}$.
- Finally, SIM' returns $(R_k, \hat{\sigma}_k)$.
3. Else, suppose the check fails, i.e., that $(k, \cdot, \text{cm}_k, \cdot, (\mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}), \cdot, \cdot) \notin Q_{\text{cm-wait}}$. For each commitment $\text{cm}_i \in \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}$, SIM' checks if it has an entry $(i, R_i, \cdot, \text{cm}_i, \cdot, \cdot) \in Q_{\text{cm}}$.
 - (a) If there exists some $\text{cm}_i \in \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}$ such that $i \in \mathcal{S} \cap \text{hon}$ and cm_i has a corresponding R_i defined in Q_{cm} , then SIM goes to Case 1.
 - (b) If there exists some $\text{cm}_j \in \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}$ such that $j \in \mathcal{S} \cap \text{cor}$ and cm_j does not have a corresponding R_j defined in Q_{cm} , then SIM' goes to Case 1.
 - (c) Otherwise, SIM' goes to Case 2.

Case 1 (Invalid Query). In this case, SIM' does the following:

1. Sample random $r_k \leftarrow \mathbb{Z}_p$ and set $R_k \leftarrow g^{r_k}$.
2. Program $Q_{\text{cm}} \leftarrow Q_{\text{cm}} \cup \{(k, R_k, r_k, \text{cm}_k, \perp, \perp)\}$.
3. Follow the protocol honestly to produce a signature $\hat{\sigma}_k$ on msg_k .
4. Add $Q_{\text{msg}} \leftarrow Q_{\text{msg}} \cup \{\text{msg}_k\}$, indicating that honest party k output a signature on msg_k .
5. Set $\text{st}_{k,\text{eid},2} \leftarrow (R_k, r_k, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \hat{\text{sk}}_k)$.
6. Update $Q_{\text{st}}[k, \text{eid}, 2] \leftarrow \text{st}_{k,\text{eid},2}$.
7. Return $(R_k, \hat{\sigma}_k)$.

Case 2 (Valid Query). In this case, SIM' does the following:

1. Sets $c \leftarrow h_{\text{ctr}}$ and $\Delta \leftarrow \text{ctr}$ and increments $\text{ctr} \leftarrow \text{ctr} + 1$.
2. For all $i \in \mathcal{S} \cap \text{hon}$, SIM' samples random $z_i \leftarrow \mathbb{Z}_p$ and computes $R_i \leftarrow g^{z_i} X_i^{-c\lambda_i}$, where λ_i is the Lagrange coefficient for party i in the set $\mathcal{S}$.
3. SIM' programs $\text{cm}_k \leftarrow H_{\text{cm}}(k, R_k)$. That is, SIM' adds the entry $Q_{\text{cm}} \leftarrow Q_{\text{cm}} \cup \{(k, R_k, \perp, \text{cm}_k, c\lambda_k, z_k)\}$. Additionally, for all $i \in \mathcal{S} \cap \text{hon}, i \neq k$, SIM' adds the entry $Q_{\text{cm-wait}} \leftarrow Q_{\text{cm-wait}} \cup \{(i, R_i, \text{cm}_i, z_i, (\mathcal{S}, m, \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}), \lambda_i c, c)\}$.
4. SIM' follows the protocol honestly to produce a signature $\hat{\sigma}_k$ on msg_k , where $\text{msg}_k = (k, \text{cm}_k, R_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}})$, and adds $Q_{\text{msg}} \leftarrow Q_{\text{msg}} \cup \{\text{msg}_k\}$, indicating that honest party k output a signature on msg_k .
5. It then sets $\text{st}_{k,\text{eid},2} \leftarrow (R_k, r_k, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \hat{\text{sk}}_k)$ and updates $Q_{\text{st}}[k, \text{eid}, 2] \leftarrow \text{st}_{k,\text{eid},2}$.

6. Next, if k is the only honest party in $\mathcal{S}$ then SIM' looks up $(i, R_i, \cdot, \text{cm}_i, \cdot, \cdot) \in \text{Q}_{\text{cm}}$ for all $i \in \mathcal{S}, i \neq k$. SIM' then computes $R = \prod_{\ell \in \mathcal{S}} R_\ell$. If $(\text{pk}, m, R, c', \Delta') \in \text{Q}_{\text{sig}}$ for some c', Δ' , then SIM' aborts. We refer to this bad event as $\text{H}_{\text{sig}}^{\text{GuessOneHon}}$. Otherwise, SIM' programs $c \leftarrow \text{H}_{\text{sig}}(\text{pk}, m, R)$. That is SIM' adds $\text{Q}_{\text{sig}} \leftarrow \text{Q}_{\text{sig}} \cup \{(\text{pk}, m, R, c, \Delta)\}$.
7. Finally, SIM' returns $(R_k, \hat{\sigma}_k)$.

Round 3 ($\mathcal{O}^{\text{Sign}_3}(k, \text{eid}, \{(R_i, \hat{\sigma}_i)\}_{i \in \mathcal{S}})$): In the third round of signing, each party i in the signing set $\mathcal{S}$ outputs the remaining part of its partial signature, namely z_i . When $\mathcal{A}$ queries $\mathcal{O}^{\text{Sign}_3}$ on $(k, \text{eid}, \{(R_i, \hat{\sigma}_i)\}_{i \in \mathcal{S}})$ for $k \in \text{hon}$, SIM' first performs the checks in $\mathcal{O}^{\text{Sign}_3}$ (Fig. 7) and Sign_3 (Fig. 8). If they pass, then, in particular $\text{st}_{k, \text{eid}, 2} \leftarrow (R_k, \perp, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \hat{\text{sk}}_k)$. SIM' then does the following:

1. Checks that $\text{DS.Verify}(\hat{X}_i, \text{msg}_i, \hat{\sigma}_i) = 1$ for all $i \in \mathcal{S}$, where $\text{msg}_i = (i, \text{cm}_i, R_i, \mathcal{S}, m, \{\text{cm}_j\}_{j \in \mathcal{S}})$, and returns $\perp$ if not.
2. Checks whether $\text{cm}_i = \text{H}_{\text{cm}}(i, R_i)$ for all $i \in \mathcal{S}$ and returns $\perp$ if not.
3. Looks up $\text{msg}_k = (k, \text{cm}_k, R_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}) \in \text{Q}_{\text{msg}}$ and aborts if no such entry exists. We refer to this bad event as ForgeDS_k .
4. Check that there exists cm_i such that $\text{msg}_i = (i, \text{cm}_i, R_i, \mathcal{S}, m, \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}) \in \text{Q}_{\text{msg}}$ for all $i \in \mathcal{S} \cap \text{hon}$ and abort if not. We refer to this bad event as ForgeDS_i .
5. Else, SIM retrieves $(k, R_k, \cdot, \text{cm}_k, \cdot, \cdot) \leftarrow \text{Q}_{\text{cm}}$ and aborts if $\cdot = \perp$. We refer to this bad event as FromInvalid_k . Otherwise SIM' retrieves z_k from $(k, R_k, \cdot, \text{cm}_k, \cdot, \cdot) \in \text{Q}_{\text{cm}}$.
6. SIM' updates $\text{Q}_{\text{st}}[k, \text{eid}, 3] \leftarrow z_k$.
7. Finally, SIM returns z_k .

Simulating Corruption Oracle Queries. $\mathcal{A}$ may at any time corrupt an honest party k by querying $\mathcal{O}^{\text{Corrupt}}(k)$. Upon receiving a corruption query, SIM' first checks that $k \in \text{hon}$, returning $\perp$ if not. Otherwise, SIM' queries its DL oracle $\mathcal{O}^{\text{dl}}$ on $X_k = g^{f(k)}$ (with representation $(1, k, \dots, k^t)$) to obtain the secret key $x_k = f(k)$. Then, for each $\text{st}_{k, \text{eid}, \cdot}$, SIM' chooses R_k and r_k as follows:

1. If there exists R_k, r_k such that $(k, R_k, r_k, \text{cm}_k, \cdot, \cdot) \in \text{Q}_{\text{cm}}$, then SIM' sets R_k and r_k accordingly. SIM' updates $\hat{\text{sk}}_k$ to sk_k and sets $\text{st}_{k, \text{eid}, 2} \leftarrow (R_k, r_k, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \text{sk}_k)$ and $\text{st}_{k, \text{eid}, 1} \leftarrow (\text{cm}_k, R_k, r_k, \text{sk}_k)$.
2. If there exists R_k, c_k, z_k such that $(k, R_k, \perp, \text{cm}_k, c_k, z_k) \in \text{Q}_{\text{cm}}$, then SIM' sets R_k accordingly and sets r_k as:

$$r_k = z_k - c_k x_k$$

SIM' updates $\hat{\text{sk}}_k$ to sk_k and sets $\text{st}_{k, \text{eid}, 2} \leftarrow (R_k, r_k, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \text{sk}_k)$ and $\text{st}_{k, \text{eid}, 1} \leftarrow (\text{cm}_k, R_k, r_k, \text{sk}_k)$.

3. If there does not exist R_k such that $(k, R_k, \cdot, \text{cm}_k, \cdot, \cdot) \in \text{Q}_{\text{cm}}$, then
 - (a) SIM' samples $r_k \leftarrow_{\$} \mathbb{Z}_p$, sets $R_k \leftarrow g^{r_k}$, and programs $\text{cm}_k \leftarrow \text{H}_{\text{cm}}(k, R_k)$ by adding $(k, R_k, r_k, \text{cm}_k, \cdot, \cdot)$ to Q_{cm} .

(b) For each $R'_k, z'_k, (\mathcal{S}', m', \{\text{cm}'_i\}_{i \in \mathcal{S}}), c'_k, c'$ with

$$(k, R'_k, \text{cm}_k, z'_k, (\mathcal{S}', m', \{\text{cm}'_i\}_{i \in \mathcal{S}}), c'_k, c') \in \mathcal{Q}_{\text{cm-wait}}$$

SIM' tests if there exists $\{R'_i\}_{i \in \mathcal{S}', i \neq k}$ such that $(i, R'_i, \cdot, \text{cm}'_i, \cdot, \cdot) \in \mathcal{Q}_{\text{cm}}$ for all $i \in \mathcal{S}', i \neq k$. If the check holds, then SIM' computes $R' \leftarrow R_k \prod_{\ell \in \mathcal{S}', \ell \neq k} R'_\ell$. If $(\text{pk}, m, R', \cdot, \cdot) \in \mathcal{Q}_{\text{sig}}$, then SIM' aborts. We refer to this bad event as $\text{H}_{\text{sig}} \text{Guess}_{\text{LastHonCor}}$. Otherwise, SIM' programs $c' \leftarrow \text{H}_{\text{sig}}(\text{pk}, m, R')$. That is, SIM' adds $\mathcal{Q}_{\text{sig}} \leftarrow \mathcal{Q}_{\text{sig}} \cup \{(\text{pk}, m, R', c', \Delta')\}$.

(c) SIM' updates $\hat{\text{sk}}_k$ to sk_k and sets $\text{st}_{k, \text{eid}, 1} \leftarrow (\text{cm}_k, R_k, r_k, \text{sk}_k)$.

4. Else, if there exists R_k such that $(k, R_k, \perp, \text{cm}_k, \perp, \perp) \in \mathcal{Q}_{\text{cm}}$, then SIM' aborts. We refer to this bad event as $\text{H}_{\text{cm}} \text{Guess}$.

Finally, SIM' sets $\text{st}_k \leftarrow \mathcal{Q}_{\text{st}}[k, \cdot, \cdot]$, and outputs $(x_k, \hat{x}_k, \text{st}_k)$.

Analysis of SIM' 's Simulation.

SIM' 's simulation of H_{sig} . SIM' 's simulation of H_{sig} is perfect because each c set in $\mathcal{O}^{\text{Sign}_2}$, Step 3, Case 2, Item 1, is only added to $\mathcal{Q}_{\text{sig}}$ once. Indeed, c can be added to $\mathcal{Q}_{\text{sig}}$ either:

1. In $\mathcal{O}^{\text{Sign}_2}$, Step 2 (b): When $(k, R_k, \text{cm}_k, z_k, (\mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}), c_k, c) \in \mathcal{Q}_{\text{cm-wait}}$, H_{sig} is only programmed if there exists $\{R_i\}_{i \in \mathcal{S}, i \neq k}$ such that $(i, R_i, \cdot, \text{cm}_i, \cdot, \cdot) \in \mathcal{Q}_{\text{cm}}$ for all $i \in \mathcal{S}, i \neq k$.
2. In $\mathcal{O}^{\text{Sign}_2}$, Step 3, Case 2, Item 6. When k is the only honest party in $\mathcal{S}$ then we have $(i, R_i, \cdot, \text{cm}_i, \cdot, \cdot) \in \mathcal{Q}_{\text{cm}}$ for all $i \in \mathcal{S}, i \neq k$.
3. In $\mathcal{O}^{\text{Corrupt}}$, Step 3 (b). When $(k, R'_k, \text{cm}_k, z'_k, (\mathcal{S}', m', \{\text{cm}'_i\}_{i \in \mathcal{S}}), c'_k, c') \in \mathcal{Q}_{\text{cm-wait}}$, H_{sig} is only programmed if there exists $\{R'_i\}_{i \in \mathcal{S}', i \neq k}$ such that $(i, R'_i, \cdot, \text{cm}'_i, \cdot, \cdot) \in \mathcal{Q}_{\text{cm}}$ for all $i \in \mathcal{S}', i \neq k$.

Thus, we see that c from $\mathcal{Q}_{\text{cm-wait}}$ is only added to $\mathcal{Q}_{\text{sig}}$ when the final $\text{cm}_i \in \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}$ is added to $\mathcal{Q}_{\text{cm}}$.

SIM' 's simulation of KeyGen and $\mathcal{O}^{\text{Sign}_1}$. SIM' 's simulation of key generation and $\mathcal{O}^{\text{Sign}_1}$ is perfect and no abort conditions occur.

SIM' 's simulation of $\mathcal{O}^{\text{Sign}_2}$. SIM' 's simulation of $\mathcal{O}^{\text{Sign}_2}$ is perfect. SIM' responds if and only if $\text{st}_{k, \text{eid}, 1}$ exists as in Game 2. The signatures $\hat{\sigma}_k$ verify on msg_k . Before revealing an honest R_k , SIM always programs H_{cm} such that $\text{cm}_k = \text{H}_{\text{cm}}(k, R_k)$, as would be the case in Game 2. In the real $\mathcal{O}^{\text{Sign}_2}$, the R_k is chosen as a uniformly random group element (during $\mathcal{O}^{\text{Sign}_1}$ but only revealed now). In the simulation, R_k is chosen either:

1. In Step 2 (a). Here, the R_k has previously been randomized by some unique z_k during an alternative $\mathcal{O}^{\text{Sign}_2}$ query that went to Step 3, Case 2, Item 3.
2. In Step 3, Case 1, Item 2. Here, the R_k is sampled uniformly at random.
3. In Step 3, Case 2, Item 4. Here the R_k is randomized by z_k .

It remains to show that H_{sigGuess} and $H_{\text{sigGuessOneHon}}$ occur with maximum probability $\frac{q}{p}$. Indeed, SIM checks to see if there is a defined entry $(\text{pk}, m, R, c', \Delta') \in Q_{\text{sig}}$ for some c', Δ' . In both cases, the probability that $\mathcal{A}$ guesses R in some hash query is bounded by $\frac{q}{p}$ because R is randomized R_k , which we have seen is distributed uniformly at random.

SIM' 's simulation of $\mathcal{O}^{\text{Sign}_3}$. SIM' 's simulation of $\mathcal{O}^{\text{Sign}_3}$ is perfect.

Recall SIM' is required to respond only to valid queries $(k, \text{eid}, \{(R_i, \hat{\sigma}_i)\}_{i \in \mathcal{S}})$, where $\text{st}_{k, \text{eid}, 2} = (R_k, \perp, \text{cm}_k, \hat{\sigma}_k, \mathcal{S}, m, \{\text{cm}_i\}_{i \in \mathcal{S}}, \hat{\text{sk}}_k)$.

For all $i \in \mathcal{S} \cap \text{hon}$ we have $(i, \text{cm}_i, R_i, \mathcal{S}, m, \{\text{cm}_\ell\}_{\ell \in \mathcal{S}}) \in Q_{\text{msg}}$. This means that R_i was returned during an $\mathcal{O}^{\text{Sign}_2}$ query with respect to the same $(\mathcal{S}, m, \{\text{cm}_\ell\}_{\ell \in \mathcal{S}})$. Thus, there exists $(i, R_i, \perp, \text{cm}_i, c_i, z_i) \in Q_{\text{cm}}$ for all $i \in \mathcal{S} \cap \text{hon}$ such that $R_i = g^{z_i} X_i^{-c_i}$.

For all $j \in \mathcal{S} \cap \text{cor}$, we have that there exists $(j, R_j, \cdot, \text{cm}_j, \cdot, \cdot) \in Q_{\text{cm}}$. Each corrupt R_j 's hashes to a unique cm_j .

Thus, some $i' \in \mathcal{S}$ must have been the last value $(i', R_{i'}, \cdot, \text{cm}_{i'}, \cdot, \cdot)$ added to Q_{cm} . Upon this occurring, we have that $(\text{pk}, m, R, c, \Delta)$ was added to Q_{sig} with $R = \prod_{\ell \in \mathcal{S}} R_\ell$ and with $c_i = \lambda_i c$ for all $i \in \mathcal{S} \cap \text{hon}$.

Thus $R_k = g^{z_k} X_k^{-c \lambda_k}$ and the response is correctly distributed.

ForgeDS_k and ForgeDS_i cannot occur unless Game 1 also would abort. FromInvalid_k occurs if some honest R_i is computed in an $\mathcal{O}^{\text{Sign}_2}$ query during Case 1 (Invalid Query). By the same argument as in the proof of Theorem 1, this occurs with maximum probability $\frac{q}{p}$.

SIM' 's simulation of $\mathcal{O}^{\text{Corrupt}}$. SIM' 's simulation of $\mathcal{O}^{\text{Corrupt}}$ is perfect.

1. If there exists R_k, r_k such that $(k, R_k, r_k, \text{cm}_k, \cdot, \cdot) \in Q_{\text{cm}}$, then R_k and r_k were chosen during Case 1. This means that r_k is randomly distributed and $R_k = g^{r_k}$.
2. If there exists R_k, c_k, z_k such that $(k, R_k, \perp, \text{cm}_k, c_k, z_k) \in Q_{\text{cm}}$, then R_k, c_k and z_k were chosen during Case 2. This means that $R_k \leftarrow g^{z_k} X_k^{-c \lambda_k}$, $c_k = c \lambda_k$ and thus:

$$R_k = g^{z_k - x_k c_k} = g^{r_k}$$

Further, r_k is randomly distributed because z_k is randomly chosen.

3. If there does not exist R_k such that $(k, R_k, \cdot, \text{cm}_k, \cdot, \cdot) \in Q_{\text{cm}}$, then by design $R_k = g^{r_k}$ for random r_k , and $\text{cm}_k = H_{\text{cm}}(k, R_k)$.

The probability that $H_{\text{sigGuessLastHonCor}}$ occurs, i.e., $(\text{pk}, m, R', \cdot, \cdot) \in Q_{\text{sig}}$, is bounded by $\frac{q}{p}$. This is because the aggregated R' is randomized by R_k (as discussed in the analysis of H_{sig}).

The probability that H_{cmGuess} occurs is bounded by $\frac{q}{p}$. Indeed, the only way in which $(k, R_k, \perp, \text{cm}_k, \perp, \perp) \in Q_{\text{cm}}$ is if the adversary queries H_{cm} on (k, R_k) and gets response cm_k . This happens if and only if there is a collision with some honest cm_k , which occurs with maximum probability $\frac{q}{p}$.

Output. When $\mathcal{A}$ terminates with output (m^*, σ^*) , SIM' first checks that it is a valid forgery, by checking that $m^* \notin Q_m$ and $\text{Verify}(\text{pk}, m^*, \sigma^*) = 1$. If either check fails, SIM' returns 0.

Otherwise, SIM' looks up the entry $(\text{pk}, m^*, R^*, c^*, I) \in Q_{\text{sig}}$ corresponding to $\mathcal{A}$'s forgery. It sets $\text{out} \leftarrow \sigma^*$. SIM' then outputs (I, out) .

Difference between Game 2 and Game 1. Let W_2 be the event that $\mathcal{A}$ wins Game 2. The only additional advantage in the simulation by SIM' over Game 1 is given by bad events ROCollision , $\text{H}_{\text{sig}}\text{Guess}$, FromInvalid_k , $\text{H}_{\text{sig}}\text{Guess}_{\text{OneHon}}$, $\text{H}_{\text{sig}}\text{Guess}_{\text{LastHonCor}}$, $\text{H}_{\text{cm}}\text{Guess}$ (recalling that ForgeDS_k and ForgeDS_i cannot occur). As such,

$$\begin{aligned} |\Pr[W_1] - \Pr[W_2]| &= \Pr[\text{ROCollision}] + \Pr[\text{H}_{\text{sig}}\text{Guess}] + \Pr[\text{FromInvalid}_k] \\ &\quad + \Pr[\text{H}_{\text{sig}}\text{Guess}_{\text{OneHon}}] + \Pr[\text{H}_{\text{sig}}\text{Guess}_{\text{LastHonCor}}] + \Pr[\text{H}_{\text{cm}}\text{Guess}] \\ &= \frac{q^2}{2p} + \frac{5q}{p} \end{aligned} \tag{15}$$

Reduction $\mathcal{B}'_2$ Against AOMDL. Finally, we construct our PPT reduction $\mathcal{B}'_2$ against the AOMDL assumption (Fig. 5). Specifically, we show how $\mathcal{B}'_2$ uses Fork^{SIM} as a subroutine such that:

$$\begin{aligned} \text{Adv}_{\mathcal{B}'_2}^{t+1\text{-aomdl}}(\kappa) &\geq \text{accept}(\text{Fork}^{\text{SIM}'}(X)) - 2\Pr[\text{ROCollision}] - 2\Pr[\text{H}_{\text{sig}}\text{Guess}] \\ &\quad - 2\Pr[\text{FromInvalid}_k] - 2\Pr[\text{H}_{\text{sig}}\text{Guess}_{\text{OneHon}}] \\ &\quad - 2\Pr[\text{H}_{\text{sig}}\text{Guess}_{\text{LastHonCor}}] - 2\Pr[\text{H}_{\text{cm}}\text{Guess}] \end{aligned} \tag{16}$$

where X is the instance given by the AOMDL challenger.

Initialization. $\mathcal{B}'_2$ is initialized by the AOMDL challenger with input the group description $(\mathbb{G}, p, g)$ and $t + 1$ AOMDL challenges $(Y_0, \dots, Y_t) \in \mathbb{G}^{t+1}$. $\mathcal{B}'_2$ has access to a discrete logarithm oracle $\mathcal{O}^{\text{dl}}$, which it may query up to t times. $\mathcal{B}'_2$ aims to output $(y_0, \dots, y_t)$ such that $Y_i = g^{y_i}$ for all $0 \leq i \leq t$, with only t queries to its DL oracle.

To simulate $\mathcal{O}^{\text{dl}}$ queries by SIM' , $\mathcal{B}'_2$ initializes a table Q_{dl} , which it uses to cache $\mathcal{O}^{\text{dl}}$ queries made by SIM' and responses from $\mathcal{O}^{\text{dl}}$.

Execution. $\mathcal{B}'_2$ then runs the general forking algorithm $\text{Fork}^{\text{SIM}'}(X)$ as described in Figure 9, on the simulator algorithm SIM' and its instance X .

When SIM' queries $\mathcal{O}^{\text{dl}}$, $\mathcal{B}'_2$ queries its own oracle $\mathcal{O}^{\text{dl}}$. $\mathcal{B}'_2$ caches the request and response in Q_{dl} , and then returns the response to SIM' .

With non-negligible probability lower-bounded by Equation 6, $\text{Fork}^{\text{SIM}'}(X)$ will output the accepting answer $(I, \text{out}, \text{out}')$, such that:

$$\begin{aligned} h_I &= c^*, h'_I = c^{**}, \\ \sigma^* &\leftarrow \text{out}, \sigma^{**} \leftarrow \text{out}' \\ (R^*, z^*) &\leftarrow \sigma^*, (R^*, z^{**}) \leftarrow \sigma^{**} \end{aligned}$$

Extracting the Discrete Logarithm of Y_0 . We show that $\mathcal{B}'_2$ can extract the discrete logarithm of Y_0 from $\mathcal{A}$'s two valid forgeries. We assume without loss of generality that $\mathcal{A}$ queries H_{sig} on (pk, R^*, m^*) on its forgery for each of its executions.

With overwhelming probability, $c^* \neq c^{**}$, and $\mathcal{B}'_2$ can solve for $y_0 = \frac{z^* - z^{**}}{c^* - c^{**}}$. If $\mathcal{B}'_2$ extracts y_0 , then we use this to extract a full AOMDL solution as follows.

Extracting an AOMDL Solution. The reduction $\mathcal{B}'_2$ must now extract the remaining $y_1, \dots, y_t$ such that $Y_i = g^{y_i}$. Assume without loss of generality that $\mathcal{A}$ makes t corruptions over the two iterations. (If not, SIM' can corrupt the remaining number at the end, by querying its DL oracle until it reaches t secret keys.) Recall that SIM' sets $X_i = Y_0 Y_1^{i_1} \dots Y_t^{i_t}$, and made t $\mathcal{O}^{\text{dl}}$ queries $g^{x_{i_1}}, \dots, g^{x_{i_t}}$:

$$\begin{aligned} g^{x_{i_1}} &= Y_0 Y_1^{i_1} \dots Y_t^{i_1^t} \\ &\vdots \\ g^{x_{i_t}} &= Y_0 Y_1^{i_t} \dots Y_t^{i_t^t} \end{aligned}$$

Recall also that $\text{pk} = Y_0$. This forms the following system of linear equations:

$$\begin{aligned} x &= y_0 \\ x_{i_1} &= y_0 + i_1 y_1 \dots + i_1^t y_t \\ &\vdots \\ x_{i_t} &= y_0 + i_t y_1 \dots + i_t^t y_t \end{aligned}$$

Equivalently,

$$\begin{pmatrix} x \\ x_{i_1} \\ \vdots \\ x_{i_t} \end{pmatrix} = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 1 & i_1 & \dots & i_1^t \\ \vdots & \vdots & \ddots & \vdots \\ 1 & i_t & \dots & i_t^t \end{pmatrix} \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_t \end{pmatrix}$$

$\mathcal{B}'_2$ knows all of the values on the left-hand side. The matrix

$$V = \begin{pmatrix} 1 & 0 & \dots & 0 \\ 1 & i_1 & \dots & i_1^t \\ \vdots & \vdots & \ddots & \vdots \\ 1 & i_t & \dots & i_t^t \end{pmatrix}$$

is a Vandermonde matrix and is therefore invertible. Thus, $\mathcal{B}'_2$ can solve for $(y_0, y_1, \dots, y_t)$ and win the $t + 1$ -aomdl game.

Finishing the Proof. Combining Equations 6, 13, 14, 15, and 16 gives Equation 4. This completes the proof. $\square$

C Proof of Adaptive Security up to t Corruptions

We now prove our strongest result: that **Sparkle+** is secure against t adaptive corruptions. In particular, if $t + 1$ parties engage in signing, all but one of them could be malicious and the unforgeability of **Sparkle+** would still hold. We prove this result under the EUF-CMA security of **DS** and under the AOMDL assumption (Fig. 5) in the AGM with random oracles. For simplicity, we assume that the signature scheme used, if it is group based, is implemented over a different group than the threshold signature.

Proof. (of Theorem 3.) We prove the theorem by a sequence of games.

Game 0. This is the adaptive unforgeability game as defined in Figure 7, instantiated with **Sparkle+**.

Let $\mathcal{A}$ be an algebraic PPT adversary attempting to break the adaptive unforgeability of **Sparkle+** (Fig. 7) that makes up to q_H queries to H_{cm} and H_{sig} , and q_S queries to its signing oracles. $\mathcal{A}$ is allowed to make t adaptive corruptions. We assume, without loss of generality, that $\mathcal{A}$ makes a query to H_{sig} on its forgery (pk, m^*, R^*) . Then, let $q = q_H + q_S + 1$.

Let W_0 be the event that $\mathcal{A}$ wins Game 0. Then, the advantage of $\mathcal{A}$ is:

$$\text{Adv}_{\mathcal{A}, \text{Sparkle}^+}^{\text{adp-TS-EUF-CMA}}(\kappa, \text{frac} = 1) = |\Pr[W_0]| \quad (17)$$

Here, $\text{frac} = 1$ to allow $\mathcal{A}$ to corrupt a full $\text{frac} \cdot t = t$ parties.

Game 1. The changes from Game 0 to Game 1 are the same as in the proof of Theorem 1.

Reduction $\mathcal{B}_1''$ to EUF-CMA Security of DS. We now define a reduction $\mathcal{B}_1''$ against the EUF-CMA security of **DS** that uses $\mathcal{A}$ as a subroutine. $\mathcal{B}_1''$ is identical to $\mathcal{B}_1'$ in the proof for Theorem 2. The only difference is the probability of **Event**₂ occurring. At the end of the game, there are $(n - t)$ non-corrupted parties. The probability that τ is one of them (i.e., that **Event**₂ does not occur) is $\frac{(n-t)}{n}$.

Difference between Game 0 and Game 1. If **Event**₁ does not occur, then Game 0 and Game 1 are identical. If **Event**₁ occurs, then the probability that $k = \tau$ is at least $\frac{1}{n}$. The probability that **Event**₂ does not occur, i.e., τ is not corrupted, is greater than $\frac{(n-t)}{n}$.

The additional advantage for $\mathcal{A}$ in Game 1, assuming that **Event**₁ occurs for participant τ , is upper-bounded by the advantage that $\mathcal{B}_1'$ wins its EUF-CMA game against **DS**, and the probability that **Event**₂ does not occur.

Let W_1 be the event that $\mathcal{A}$ wins Game 1. Then,

$$|\Pr[W_0] - \Pr[W_1]| \leq \frac{n^2}{(n-t)} \text{Adv}_{\mathcal{B}_1'', \text{DS}}^{\text{EUF-CMA}}(\kappa) \quad (18)$$

Reduction $\mathcal{B}_2''$ against AOMDL. We now construct a PPT reduction $\mathcal{B}_2''$ against the AOMDL assumption (Fig. 5). Specifically, we show how $\mathcal{B}_2''$ uses the algebraic adaptive unforgeability adversary $\mathcal{A}$ as a subroutine such that:

$$\begin{aligned} \text{Adv}_{\mathcal{B}_2''}^{t+1\text{-aomdl}}(\kappa) &\geq \text{Adv}_{\mathcal{A}, \text{Sparkle}^+}^{\text{adp-TS-EUF-CMA}}(\kappa, 1) - \Pr[\text{ROCollision}] - \Pr[\text{H}_{\text{sig}} \text{Guess}] \\ &\quad - \Pr[\text{FromInvalid}_k] - \Pr[\text{H}_{\text{sig}} \text{Guess}_{\text{OneHon}}] \\ &\quad - \Pr[\text{H}_{\text{sig}} \text{Guess}_{\text{LastHonCor}}] - \Pr[\text{H}_{\text{cm}} \text{Guess}] \end{aligned} \quad (19)$$

where X is the instance given by the AOMDL challenger.

Initialization. $\mathcal{B}_2''$ is initialized by the AOMDL challenger with input the group description $(\mathbb{G}, p, g)$ and $t+1$ AOMDL values $(Y_0, \dots, Y_t) \in \mathbb{G}^{t+1}$. $\mathcal{B}_2''$ has access to a discrete logarithm oracle $\mathcal{O}^{\text{dl}}$, which it may query up to t times. $\mathcal{B}_2''$ aims to output $(y_0, \dots, y_t)$ such that $Y_i = g^{y_i}$ for all $0 \leq i \leq t$, with only t queries to its DL oracle.

Simulation. The simulation by $\mathcal{B}_2''$ is the same as that of SIM' in Game 2 for Theorem 2, with the exception that SIM' takes input values $\{h_1, \dots, h_q\}$, whereas these should be sampled by $\mathcal{B}_2''$. As such, the difference between the simulation by $\mathcal{B}_2''$ and Game 1 is likewise upper-bounded by Equation 15.

When $\mathcal{A}$ terminates with output (m^*, σ^*) , $\mathcal{B}_2''$ first checks that it is a valid forgery, by checking that $m^* \notin \mathcal{Q}_m$ and $\text{Verify}(\text{pk}, m^*, \sigma^*) = 1$. If either check fails, $\mathcal{B}_2''$ returns 0. Otherwise, $\mathcal{B}_2''$ looks up the entry $(\text{pk}, m^*, R^*, c^*, \Delta^*) \in \mathcal{Q}_{\text{sig}}$ corresponding to $\mathcal{A}$'s forgery.

The probability that $\mathcal{A}$ is successful and outputs a valid forgery but $\mathcal{B}_2''$ aborts is upper bounded by the difference between Game 0, Game 1, and the probability of additional bad events $\Pr[\text{ROCollision}]$, $\Pr[\text{H}_{\text{sig}} \text{Guess}]$, $\Pr[\text{ForgeDS}_k]$, $\Pr[\text{ForgeDS}_i]$, $\Pr[\text{FromInvalid}_k]$, $\Pr[\text{H}_{\text{sig}} \text{Guess}_{\text{OneHon}}]$, $\Pr[\text{H}_{\text{sig}} \text{Guess}_{\text{LastHonCor}}]$, $\Pr[\text{H}_{\text{cm}} \text{Guess}]$, as given in Equation 18 and Equation 19.

Extracting the Discrete Logarithm of Y_0 . $\mathcal{A}$'s forgery verifies as:

$$R^* = g^{z^*} \text{pk}^{-c^*} \quad (20)$$

where $c^* = \text{H}_{\text{sig}}(\text{pk}, m^*, R^*)$. On the other hand, when $\mathcal{A}$ made its query $\text{H}_{\text{sig}}(\text{pk}, m^*, R^*)$, it provided a representation of R^* in terms of all of the group elements it had seen so far, namely $(g, \text{pk}, X_1, \dots, X_n, \{R_{i,1}, \dots, R_{i,n}\}_{i \in [q_S]})$, where $\{R_{i,k}\}_{i \in [q_S]}$ are the honest nonces returned by $\mathcal{O}^{\text{Sign}_1}$ over the q_S signing queries that $\mathcal{A}$ makes. We assume without loss of generality that $\mathcal{A}$ completes every signing session. (Otherwise, $\mathcal{B}_2''$ can perform any unmet $\mathcal{O}^{\text{Sign}_2}$ and $\mathcal{O}^{\text{Sign}_3}$ queries itself.) Thus, $\mathcal{A}$ provided $(\gamma^*, \xi^*, \xi_1^*, \dots, \xi_n^*, \{\rho_{i,1}^*, \dots, \rho_{i,n}^*\}_{i \in [q_S]})$ such that:

$$R^* = g^{\gamma^*} \text{pk}^{\xi^*} X_1^{\xi_1^*} \dots X_n^{\xi_n^*} \prod_{i=1}^{q_S} R_{i,1}^{\rho_{i,1}^*} \dots R_{i,n}^{\rho_{i,n}^*}$$

where $\rho_{i,k}^* = 0$ for k not in the signing set for query i . Each $R_{i,k}$ verifies as $R_{i,k} = g^{z_{i,k}} X_k^{-c_i \lambda_{i,k}}$, where $c_i = H_{\text{sig}}(\text{pk}, m_i, R_i)$. Thus,

$$R^* = g^{\gamma^*} \text{pk}^{\xi^*} X_1^{\xi_1^*} \dots X_n^{\xi_n^*} \prod_{i=1}^{qS} (g^{z_{i,1}} X_1^{-c_i \lambda_{i,1}})^{\rho_{i,1}^*} \dots (g^{z_{i,n}} X_n^{-c_i \lambda_{i,n}})^{\rho_{i,n}^*}$$

Equating this with Eq. (20), we have:

$$g^{z^*} \text{pk}^{-c^*} = g^{\gamma^*} \text{pk}^{\xi^*} X_1^{\xi_1^*} \dots X_n^{\xi_n^*} \prod_{i=1}^{qS} (g^{z_{i,1}} X_1^{-c_i \lambda_{i,1}})^{\rho_{i,1}^*} \dots (g^{z_{i,n}} X_n^{-c_i \lambda_{i,n}})^{\rho_{i,n}^*}$$

Rearranging, we have:

$$\begin{aligned} g^{z^*} g^{-\gamma^*} \prod_{i=1}^{qS} g^{-z_{i,1} \rho_{i,1}^*} \dots g^{-z_{i,n} \rho_{i,n}^*} \\ = \text{pk}^{c^*} \text{pk}^{\xi^*} X_1^{\xi_1^*} \dots X_n^{\xi_n^*} \prod_{i=1}^{qS} (X_1^{-c_i \lambda_{i,1}})^{\rho_{i,1}^*} \dots (X_n^{-c_i \lambda_{i,n}})^{\rho_{i,n}^*} \end{aligned} \quad (21)$$

Let $\eta^* = z^* - \gamma^* - \sum_{i=1}^{qS} (z_{i,1} \rho_{i,1}^* + \dots + z_{i,n} \rho_{i,n}^*)$ and $\zeta_k^* = \xi_k^* - \sum_{i=1}^{qS} c_i \lambda_{i,k} \rho_{i,k}^*$ for all $k \in [n]$. Then Eq. (21) can be rewritten as:

$$g^{\eta^*} = \text{pk}^{c^* + \xi^*} X_1^{\zeta_1^*} \dots X_n^{\zeta_n^*} \quad (22)$$

Recall that $X_i = \text{pk} Y_1^i Y_2^{i^2} \dots Y_t^{i^t}$ for all $i \in [n]$. We have that $Y_i = \text{pk}^{L'_{0,i}} \prod_{j \in \text{cor}} g^{x_j L'_{j,i}}$ for all $i \in [t]$.⁵ Thus,

$$X_k = \text{pk} \prod_{i=1}^t \left(\text{pk}^{L'_{0,i}} \prod_{j \in \text{cor}} g^{x_j L'_{j,i}} \right)^{k^i}$$

Let $\mu_k^* = 1 + \sum_{i=1}^t L'_{0,i} k^i$ and $\nu_k^* = \sum_{i=1}^t \left(\sum_{j \in \text{cor}} x_j L'_{j,i} \right) k^i$. Then X_k can be rewritten as:

$$X_k = \text{pk}^{\mu_k^*} g^{\nu_k^*}$$

and Eq. (22) can be rewritten as:

$$g^{\eta^*} = \text{pk}^{c^* + \xi^*} \prod_{k=1}^n (\text{pk}^{\mu_k^*} g^{\nu_k^*})^{\zeta_k^*}$$

⁵ There exists a unique degree- t polynomial $f(Z) = x + y_1 Z + \dots + y_t Z^t$ such that $f(0) = x$ and $f(j) = x_j$ for all $j \in \text{cor}$. It holds that $y_i = L'_{0,i} x + \sum_{j \in \text{cor}} L'_{j,i} x_j$ for the appropriate Lagrange coefficients $L'_{0,i}, L'_{j,i}$. In general, for public key shares $X_i = \text{pk} Y_1^i Y_2^{i^2} \dots Y_t^{i^t}$ for $i \in [n]$, it holds that $Y_i = \text{pk}^{L'_{0,i}} \prod_{j \in \text{cor}} g^{x_j L'_{j,i}}$ for $i \in [t]$.

Rearranging, we have:

$$g^{\eta^* - \sum_{k=1}^n \nu_k^* \zeta_k^*} = \text{pk}^{c^* + \xi^* + \sum_{k=1}^n \mu_k^* \zeta_k^*}$$

and

$$x = \frac{\eta^* - \sum_{k=1}^n \nu_k^* \zeta_k^*}{c^* + \xi^* + \sum_{k=1}^n \mu_k^* \zeta_k^*}$$

$\mathcal{A}$ fixed R^* and thus $\eta^*, \{\nu_i^*\}_{i \in [n]}, \xi^*, \{\mu_i^*\}_{i \in [n]}$ as it queried $\text{H}_{\text{sig}}(\text{pk}, m^*, R^*)$ to receive random c^* . Thus, the denominator is nonzero with overwhelming probability and $\mathcal{B}_2''$ can solve for x .

Extracting an AOMDL Solution. As in the proof of Theorem 2, $\mathcal{B}_2''$ can solve for $(y_0, y_1, \dots, y_t)$ and win the $t + 1$ -aomdl game.

Finishing the Proof. Combining Equations 17, 18, and 19 gives Equation 5. This completes the proof. $\square$